

MCMaster UNIVERSITY · ELECTRICAL AND COMPUTER ENGINEERING

The Battery SOC Benchmark

Codebase manual

WRITTEN BY

Ahmad Ali

MASc student · platform development and evaluation infrastructure

Dr. Phillip J. Kollmeyer's battery research group
McMaster University, Hamilton, Ontario

CONTRIBUTORS

Dr. Phillip J. Kollmeyer

project lead

Ahnaf Akif Rahman

SOC estimation models and evaluation

Mina (Naguib) Nassim

data collection and the original MATLAB evaluation tool

Fauzia Khanum

data collection and dataset curation

Atjen von Liebenstein

embedded deployment and complexity of SOC estimators

Paarth Kadakia

software intern

Aidan McLean

software intern

BENCHMARK AND DATASET

P. J. Kollmeyer, M. Naguib, F. Khanum and A. Emadi, *A Blind Modeling Tool for Standardized Evaluation of Battery State of Charge Estimation Algorithms*, IEEE Transportation Electrification Conference and Expo, 2022

Dataset: *Tesla Model 3 2170 Li-ion Cell Dataset and Battery SOC Estimation Blind Modeling Tool*, Borealis, doi:10.5683/SP3/ZVTR4B

FUNDING

Natural Sciences and Engineering Research Council of Canada (NSERC)

HOW TO READ THIS BOOK

This manual describes the Battery SOC Benchmark as a complete system: the estimation problem it addresses, the evaluation methodology, the web platform through which models are submitted and ranked, and the software and infrastructure that carry out the evaluation. It is written for two audiences. Researchers, students and collaborators who need to understand what the platform does and why it is designed as it is will find each chapter self-contained at the level of its summary and figures. Developers who intend to maintain or extend the platform should read the chapters in full and open the source files listed in each chapter's summary.

The chapters build on one another and are best read in order on a first pass:

- **Chapters 1 to 3** require no programming background. They cover the state-of-charge estimation problem, the architecture of the platform in outline, and the workflow of a researcher who submits a model.
- **Chapters 4 to 9** describe the system one layer at a time: the evaluation pipeline, the scoring methodology, the data model, security, administration and infrastructure.
- **Chapter 10** is a reference for common maintenance tasks and where each is performed.
- **Chapter 11** is a suggested sequence for demonstrating the platform in ten minutes.
- **The glossary and appendix** collect terminology, configuration, limits and schema details so that the chapters remain readable.

All diagrams share one visual vocabulary, introduced in Section 2.4, so that a given kind of component looks the same on every page. Screenshots show the production site as of September 2026.

CONTENTS

1 The estimation problem

- 1.1 A quantity that cannot be measured
- 1.2 Drive-cycle data
- 1.3 How the data was produced
- 1.4 The reference state of charge
- 1.5 Why temperature matters
- 1.6 Estimation error
- 1.7 Open and withheld data
- 1.8 What the benchmark provides

2 The platform in outline

- 2.1 Five stages
- 2.2 Three programs and one rule
- An analogy: the sealed examination
- 2.3 Where the components are hosted
- 2.4 Visual conventions

3 Using the platform

- 3.1 Registration
- 3.2 Validating a package before submission
- 3.3 Submission
- 3.4 Interpreting results
- 3.5 The leaderboard
- 3.6 Co-authors and contests

4 The life of a submission

- 4.1 Overview
- 4.2 The stages in detail
- 4.3 Timing
- 4.4 States and transitions

5 Scoring methodology

- 5.1 The model interface
- 5.2 The evaluation set
- 5.3 From per-cycle errors to one score
- 5.4 Auxiliary quantities
- 5.5 Outputs

6 The data model

7 Accounts and security

- 7.1 Why verification requires a button
- 7.2 Authentication and sessions
- 7.3 Containing a hostile submission

8 Administration and monitoring

- 8.1 The administrative pages
- 8.2 Revising the weights
- 8.3 Notifications
- 8.4 The outage monitor

9 Infrastructure

- 9.1 The deployed services
- 9.2 The virtual machine
- 9.3 MATLAB in a container
- 9.4 Continuous integration
- 9.5 Running the system locally

10 Maintenance reference

11 A ten-minute demonstration

12 Glossary

13 Appendix: reference tables

CHAPTER 1

The estimation problem

In this chapter. Why the state of charge of a lithium-ion cell must be estimated rather than measured; what drive-cycle data is; how, where and by whom the dataset was produced; how the reference state of charge was established and why it can be trusted; what temperature does to a cell; how estimation error is defined; and why part of the dataset is withheld.

1.1 A quantity that cannot be measured

The **state of charge (SOC)** of a battery is the fraction of its usable capacity that remains, conventionally expressed as a percentage from 0 to 100. It is the quantity behind the range indicator of an electric vehicle, and it is an input to nearly every other decision a **battery management system (BMS)** makes: how much power may be drawn, how fast the pack may be charged, and when the vehicle must be protected from over-discharge. Yet no sensor measures it. What the BMS can observe, at rates of a few hertz, is the terminal current, the terminal voltage and the cell temperature. The state of charge must be inferred from these signals by an **estimator**, an algorithm executed continuously on the vehicle's battery controller.

WHY THE STATE OF CHARGE HAS TO BE ESTIMATED

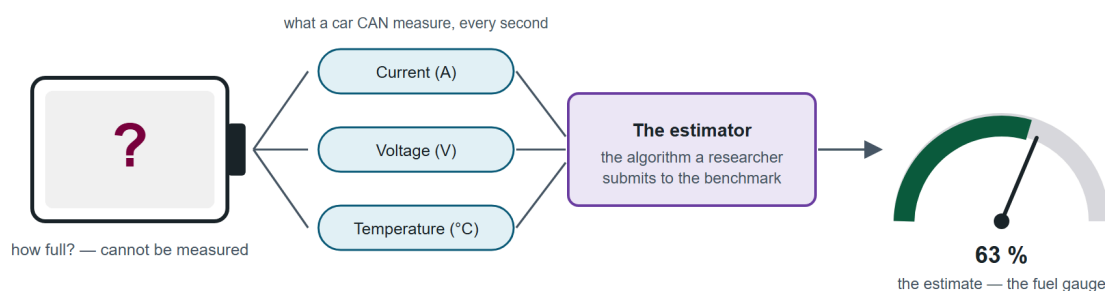


Figure 1.1 The state of charge cannot be measured directly. An estimator infers it from the three signals a battery management system can observe.

The consequences of estimation error are practical. An estimate that reads high exposes the driver to a battery that is emptier than indicated; one that reads low forces the manufacturer to reserve capacity as a safety margin, capacity the customer has paid for but can never use. Estimators are also notoriously sensitive to operating conditions: an approach that performs well at room temperature may drift severely at sub-zero

temperatures, where the cell's internal resistance rises and the relationship between voltage and state of charge flattens. A substantial research literature is therefore devoted to improving estimation methods, ranging from Coulomb counting and Kalman filtering to recurrent and transformer neural networks. This benchmark exists to evaluate those methods on a common footing.

1.2 Drive-cycle data

A **drive cycle** is a standardised speed-versus-time profile representing a particular kind of driving. Regulatory bodies have used a small set of them for decades to certify fuel economy and emissions, which makes them well characterised and reproducible. Figure 1.2 shows the four standard cycles the benchmark uses, as published by the United States Environmental Protection Agency:

- **UDDS**, the Urban Dynamometer Driving Schedule, representing stop-and-go city driving.
- **HWFET**, the Highway Fuel Economy Test, representing steady highway driving.
- **LA92** and **US06**, more aggressive profiles with higher speeds, harder acceleration and harder braking.

The four standard drive cycles as speed profiles (US EPA dynamometer schedules)

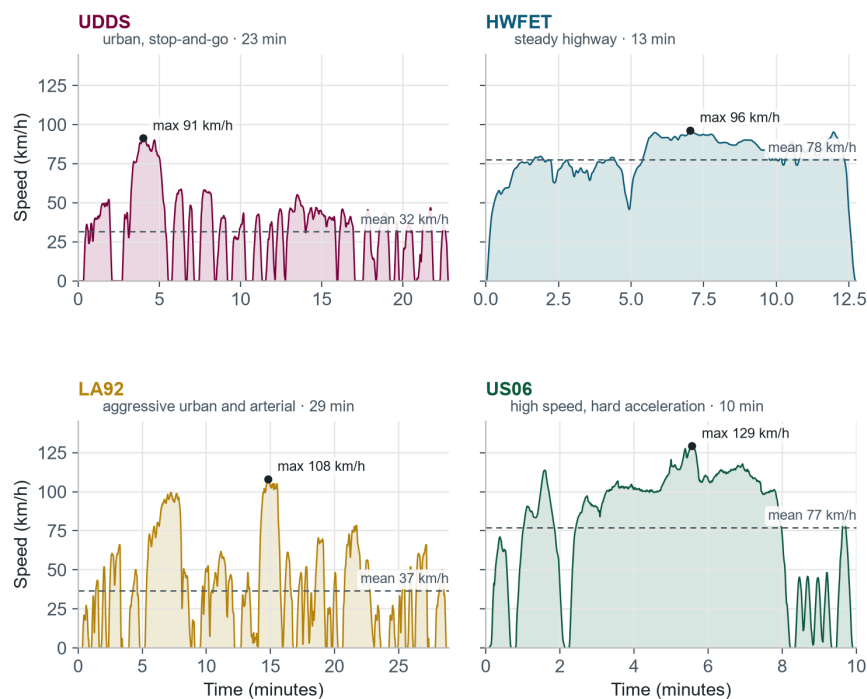


Figure 1.2 The four standard drive cycles as speed profiles. UDDS and LA92 are urban, with frequent stops; HWFET and US06 are sustained high-speed driving. Each profile was converted to the current a Model 3 cell would deliver while following it.

To these the laboratory added its own profiles. Eight **REORDERED** cycles were assembled by splicing randomly ordered segments of the four standard cycles into trips of several hours, so that a model cannot recognise a standard cycle by its shape. Two custom highway cycles, **HWCUST1** and **HWCUST2**, and two highway cycles with varying road grade, **HWGRADE1** and **HWGRADE2**, complete the set. One of each custom pair is published; the other is withheld.

A speed profile on its own says nothing about the battery. To turn it into a battery test, the laboratory used a vehicle model of a Tesla Model 3 standard-range car (50 kWh pack, 1612 kg curb mass) to compute, second by second, the current a single cell in that pack would deliver while following the profile: large discharge currents during acceleration, regenerative charging during braking, and no current at rest. Four cells were each assigned a different vehicle configuration, so that the same drive cycle produces four different current demands:

CELL	PAYLOAD ABOVE CURB MASS	CABIN CLIMATE CONTROL	REPRESENTS
m80	80 kg	on	A single occupant
m448	448 kg	on	A full passenger load
m448N	448 kg	off	A full passenger load, heating and cooling off
m1000	1000 kg	on	A full load with a trailer

Figure 1.3 shows what the resulting recording looks like for one cell over one cycle.

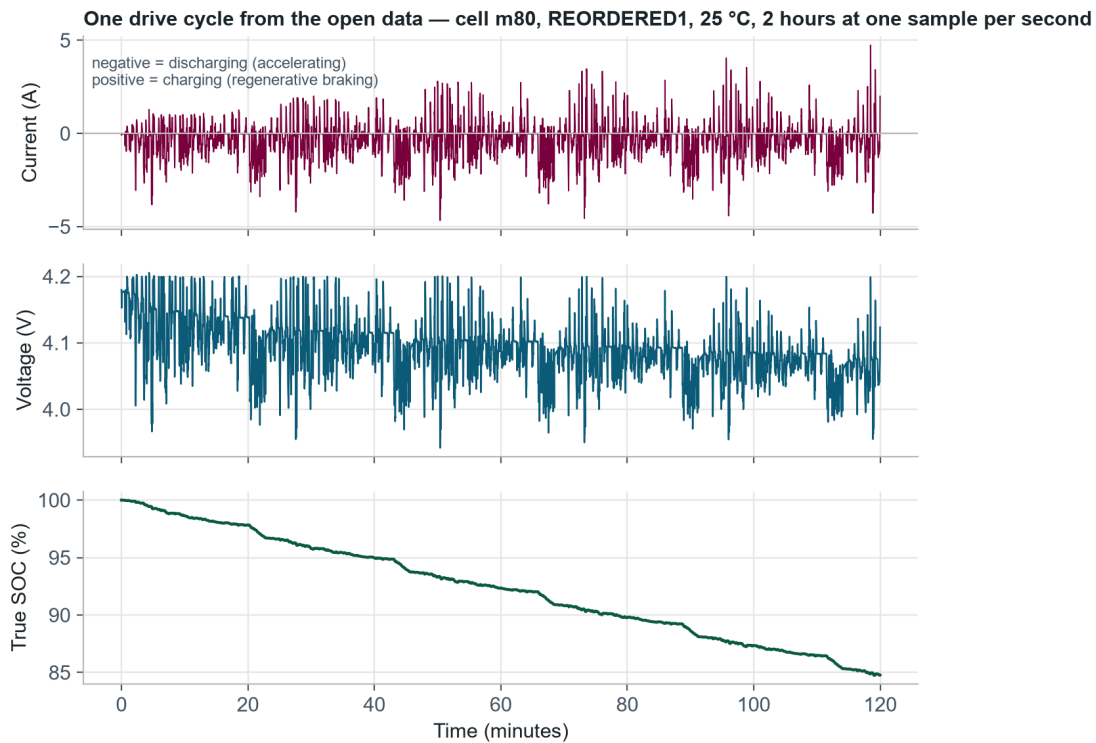


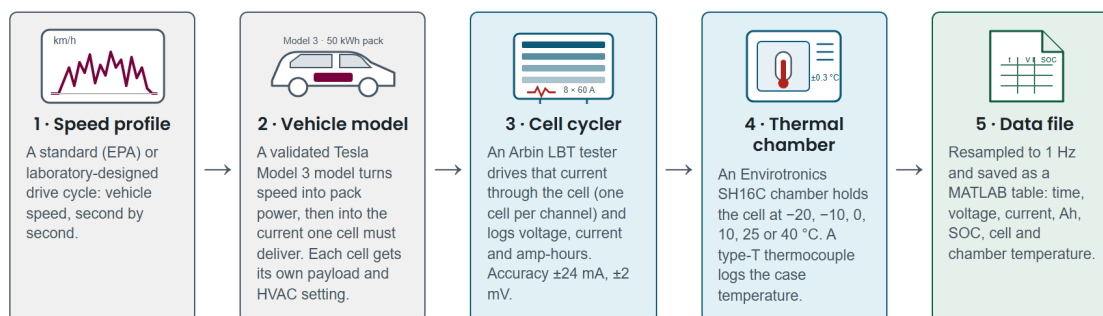
Figure 1.3 Two hours of one drive cycle from the open dataset. Current alternates between discharge and regenerative charge with every acceleration and braking event; voltage responds to it; the reference state of charge declines from full. A model receives the current, voltage and temperature and must reproduce the state-of-charge trace.

1.3 How the data was produced

The dataset was produced in the McMaster Energy Storage Laboratory at the McMaster Automotive Resource Centre in Hamilton, Ontario, in 2021. The tests were carried out by Fauzia Khanum and Mina Naguib under the supervision of Dr. Phillip Kollmeyer, and the dataset and the evaluation methodology were published by Kollmeyer, Naguib, Khanum and Ali Emadi at the IEEE Transportation Electrification Conference in 2022. Version 2 of the dataset, with corrected and additional files, was released in July 2026.

The cells are four cylindrical 2170 lithium-ion cells of nickel cobalt aluminium (NCA) chemistry, manufactured by Panasonic for Tesla and taken from a Model 3 battery pack. Their nominal capacity is 4.5 Ah, and each measured approximately 4.7 Ah at the start of testing. Figure 1.4 summarises the equipment chain.

FROM A SPEED PROFILE TO A LOGGED BATTERY TEST



Before every test the cell is returned to 25 °C and charged CC-CV to 4.2 V / 0.1 A, so every recording starts at 100 % state of charge. Tests were run hottest temperature first.

Figure 1.4 From a speed profile to a logged battery test. Every recording in the dataset passed through this chain.

Two instruments did the work. An **Arbin LBT cell cycler** with eight 60 A channels applied the computed current to each cell (one cell per channel) and logged voltage, current and accumulated amp-hours; its stated control accuracy is ± 24 mA on the 60 A range and ± 2 mV on voltage, and it can log at up to 2000 samples per second. An **Envirotronics SH16C thermal chamber** held the cells at the test temperature to within ± 0.3 °C, and a type-T thermocouple on each cell's case recorded its actual temperature. All recordings were resampled to one sample per second before publication.

The test campaign followed a fixed procedure for each cell:

1. Tests were run at 40, 25, 10, 0, -10 and -20 °C, in that order, hottest first.
2. Before every test, the cell was brought back to 25 °C and charged by the constant-current, constant-voltage method to 4.2 V with a 0.1 A cut-off, so that every recording begins from a fully charged cell.
3. At each temperature a set of **characterisation tests** was run first: a slow C/20 discharge at 40 °C to measure capacity, discharges at C/3, C/2 and 1C, a C/20 discharge-and-charge pair, and a hybrid pulse power characterisation (HPPC) test with discharge and charge pulses at every 5 to 10 % of state of charge. These are the tests from which equivalent-circuit and open-circuit-voltage models are parameterised.
4. The **drive cycles** followed: the four standard cycles, the eight REORDERED cycles and the four custom highway cycles, each driven from a full charge down to a cut-off determined from that temperature's HPPC test and a 60 kW vehicle power limit.

Between temperature rounds the cell rested while the chamber changed temperature. The whole campaign for one cell spans about eight months of instrument time, which is why the file names carry dates from April to December 2021.

1.4 The reference state of charge

Every score in the benchmark rests on the state-of-charge column in these files, so it matters how that column was obtained and how far it can be trusted. It is not a sensor reading; it is derived from two measured quantities under controlled conditions that a vehicle never enjoys.

The first quantity is the **charge removed**, which the cycler measures by integrating its own current (Coulomb counting) with an instrument whose current error is a few tens of milliamps and which is periodically re-zeroed at full charge. The second is the **capacity** of the cell, the total charge it holds. Capacity is not fixed: it depends on temperature, on the rate of discharge and on the cell's age. The laboratory therefore adopted one fixed definition, a C/20 discharge (a very gentle discharge lasting about twenty hours) performed at 40 °C, and measured it afresh at the start of every temperature round. Figure 1.5 shows one such measurement and how the reference declined over the campaign.

The capacity reference behind every state-of-charge value

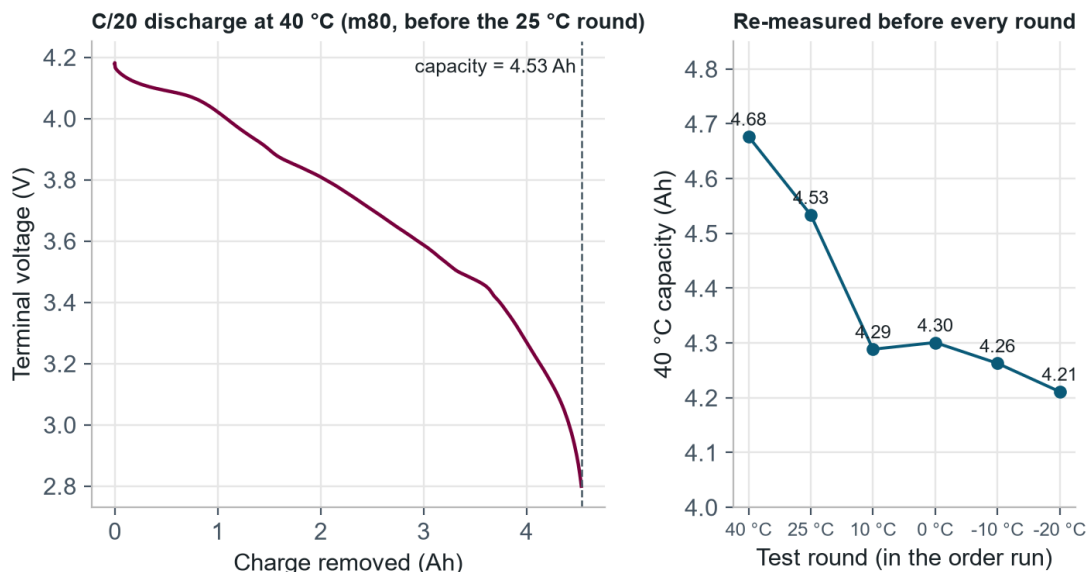


Figure 1.5 The capacity reference. Left: the C/20 discharge at 40 °C recorded before the 25 °C round, which defines 4.53 Ah as that round's capacity. Right: the same measurement repeated before every round, showing the cell losing about 10 % of its capacity over the eight-month campaign.

The state of charge in every file is then the fraction of that reference capacity still in the cell. With $I(\tau)$ the measured current in amperes (positive when charging) and Q_{ref} the reference capacity in ampere-hours,

$$\text{SOC}(t) = 1 - \frac{Q_{\text{removed}}(t)}{Q_{\text{ref}}}, \quad Q_{\text{removed}}(t) = -\frac{1}{3600} \int_0^t I(\tau) d\tau \quad (1.1)$$

Because the cell ages during a round, the reference capacity is not applied as a single number. It is interpolated linearly between the value measured at the start of the round, $Q^{(n)}$, and the value measured at the start of the next one, $Q^{(n+1)}$, across the twenty-three tests that make up a round:

$$Q_{\text{ref}, k} = Q^{(n)} + \frac{k}{22}(Q^{(n+1)} - Q^{(n)}), \quad k = 0, 1, \dots, 22 \quad (1.2)$$

For the 25 °C round of the m80 cell, for example, the reference runs from 4.530 Ah for the first test down to 4.453 Ah for the last. The final round at −20 °C has no successor, so its end value was extrapolated from the trend of the three preceding rounds.

This definition has three properties worth understanding. It is **traceable**: the raw amp-hour column is preserved in every file, so anyone can recompute the state of charge from a different capacity definition if their application calls for one. It is **consistent**: the same definition is applied to every cell, temperature and cycle, so an error of 2 % means the same thing everywhere. And it is **not something a vehicle can reproduce**: a car has no laboratory-grade current sensor, no twenty-hour capacity test and no rest periods, which is precisely why an estimator is needed, and why the benchmark's robustness tests (Chapter 5) deliberately corrupt the current signal and the starting point.

1.5 Why temperature matters

Figure 1.6 illustrates the single most important effect the dataset captures. The same cell was driven through the same REORDERED1 cycle at each of the six temperatures. At 40 °C the cell delivered 4.51 Ah before reaching the cut-off, close to its full reference capacity. At −20 °C it delivered 2.82 Ah, and the test ended with the reference state of charge still above 30 %, because the cold cell could no longer supply the vehicle's power demand.

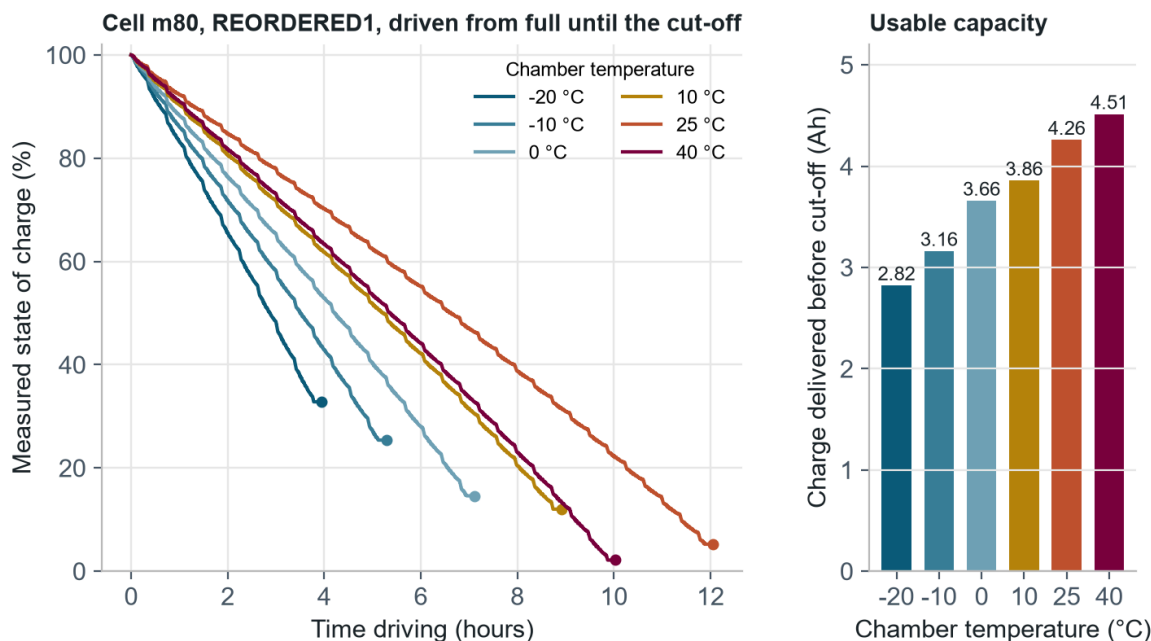


Figure 1.6 The same drive cycle on the same cell at six temperatures. Left: the reference state of charge over the trip, which ends at the power cut-off. Right: the charge delivered before cut-off at each temperature. A cold cell reaches the cut-off with a large fraction of its charge unusable.

The two panels show the two faces of the problem. The slope of each trace is the current draw, which the vehicle model held broadly similar across temperatures; the traces end early in the cold because the cell's internal resistance rises sharply, so the same power demand causes a larger voltage drop and the cut-off is reached sooner. An estimator must track state of charge correctly through this regime, where the relationship between terminal voltage and state of charge that many estimators rely on is at its weakest.

1.6 Estimation error

For every sample of every drive cycle the reference state of charge is known from the laboratory measurement described in Section 1.4. The model under test, given only current, voltage and temperature, produces its own estimate for the same sample. The difference between the two is the **estimation error**, and it is the sole quantity the benchmark measures. Every number on the leaderboard is an aggregate of it.

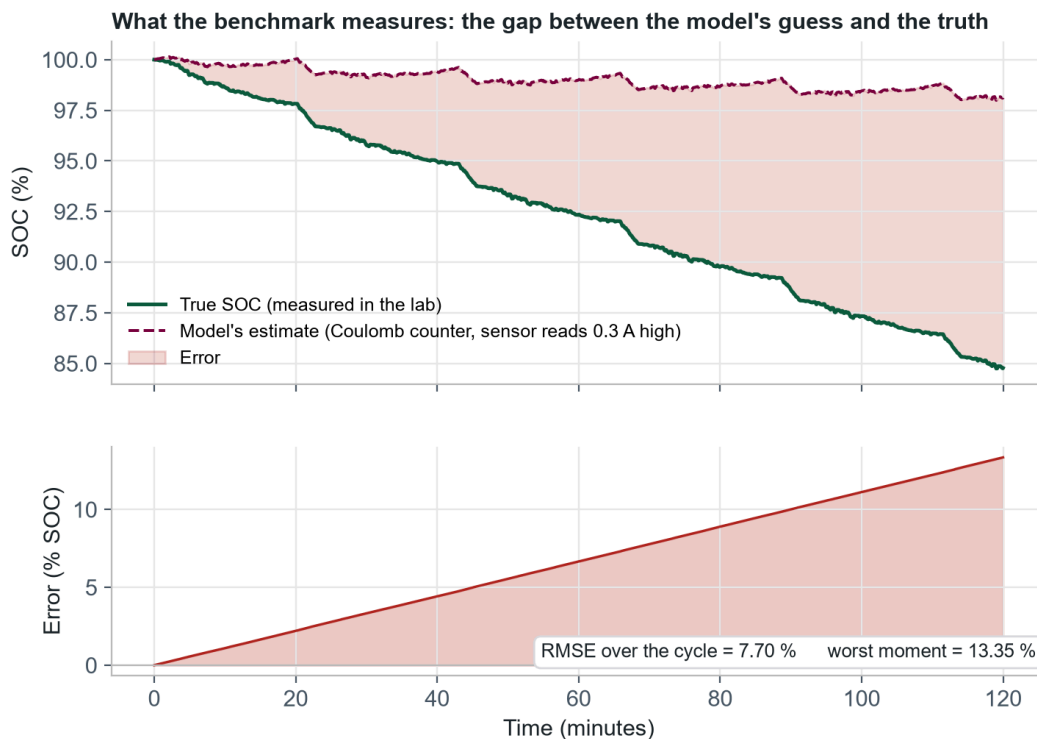


Figure 1.7 Estimation error is the difference between the model's estimate and the reference state of charge. Here a Coulomb-counting model fed a biased current measurement drifts steadily from the reference; the shaded band is what the benchmark quantifies.

The error over a cycle is summarised by its **root-mean-square error (RMSE)**. With $\widehat{\text{SOC}}_k$ the model's estimate and SOC_k the reference at sample k of a cycle of N samples,

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{k=1}^N (\widehat{\text{SOC}}_k - \text{SOC}_k)^2}, \quad \text{MAE} = \frac{1}{N} \sum_{k=1}^N |\widehat{\text{SOC}}_k - \text{SOC}_k| \quad (1.3)$$

The mean absolute error (MAE) and the maximum absolute error are reported alongside it but do not enter the score. RMSE penalises large excursions more heavily than a mean absolute error would. A model that is consistently 2 % high scores an RMSE of 2 %; a model that is exact for most of a cycle but wrong by 20 % for a few minutes scores considerably worse. This weighting is deliberate, since in a vehicle a bounded, predictable error is far more useful than an occasional large one.

The conditions under which error occurs matter as much as its magnitude. A model that is accurate at 25 °C but drifts at −20 °C is of limited use in a cold climate, and a model that is accurate only when initialised with the exact starting state of charge is of limited use in a vehicle, which does not know it. The scoring methodology therefore does not simply average error over the dataset. It reports error separately by cell, temperature and cycle type, and it includes two deliberate perturbations: a wrong initial state of charge and a constant bias on the current measurement. Chapter 5 sets out the full methodology.

1.7 Open and withheld data

Publishing the entire dataset and inviting groups to report their own results would not produce comparable numbers. Each group would select its own test cycles and its own error definition, and a model trained on published data can memorise it and appear far better than it is. The benchmark addresses both problems with a held-out evaluation set.

OPEN DATA AND HIDDEN DATA

Open data — published on Borealis

Characterisation tests and drive cycles for the m80, m448N and m1000 cells. Anyone can download it. This is what researchers build and train their models on.

Hidden data — only on the evaluation machine

The whole m448 cell, plus cycles and conditions the open data does not contain. Never published, never on the website. This is what every model is scored on, so no model can have memorised it.

A model that scores well on the open cells but badly on the hidden one has memorised the published data rather than learned the battery — and the leaderboard shows both numbers side by side.

Figure 1.8 The open dataset is for developing models; the withheld dataset is reserved for scoring them. The m448 cell exists only on the evaluation machine.

The **open dataset**, published on the Borealis research data repository under a Creative Commons Attribution licence, contains the m80, m448N and m1000 cells: every characterisation test, the eight REORDERED cycles and one of each custom highway pair at every temperature. Researchers develop and train on it without restriction. The **withheld dataset** (referred to in the code as the *blinded* data) contains the four standard cycles for every cell, the second cycle of each custom pair, the whole of the m448 cell, and the charging profiles. It has never been published and it never touches the web tier. Every submitted model is scored on the withheld data, and the leaderboard reports the withheld-cell error beside the open-cell error. A model that performs well on the open cells and poorly on the withheld one has fitted the published data rather than learned the underlying behaviour, and the comparison makes this visible.

1.8 What the benchmark provides

The benchmark provides a single, fixed evaluation: the same withheld data, the same evaluation code, the same error definition and the same published weights for every model. A researcher packages an estimator as a single Python or MATLAB function together with its parameter files, uploads the package, and receives within minutes a weighted score, a per-test breakdown, per-cycle traces and a PDF report. The score is placed on a public leaderboard where it is directly comparable with every other entry. The uploaded package is deleted as soon as it has been evaluated.

CHAPTER 2

The platform in outline

In this chapter. The five stages from open data to leaderboard, the three programs that implement them, the architectural rule that protects the withheld data, and where each component is hosted.

Files to open: [README.md](#) → [prisma/schema.prisma](#) → [src/evaluator/worker.ts](#).

2.1 Five stages

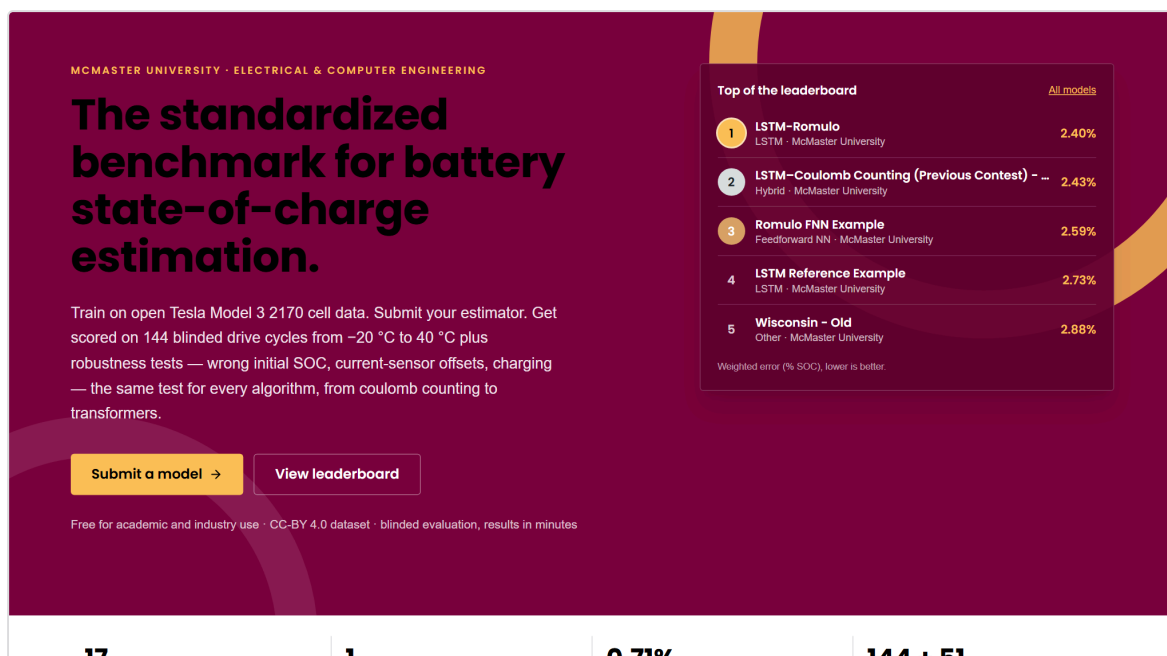


Figure 2.1 The public face of the platform: the landing page, with the top of the leaderboard.

The platform reduces to five stages, and its credibility rests on the separation between the first and the fourth:

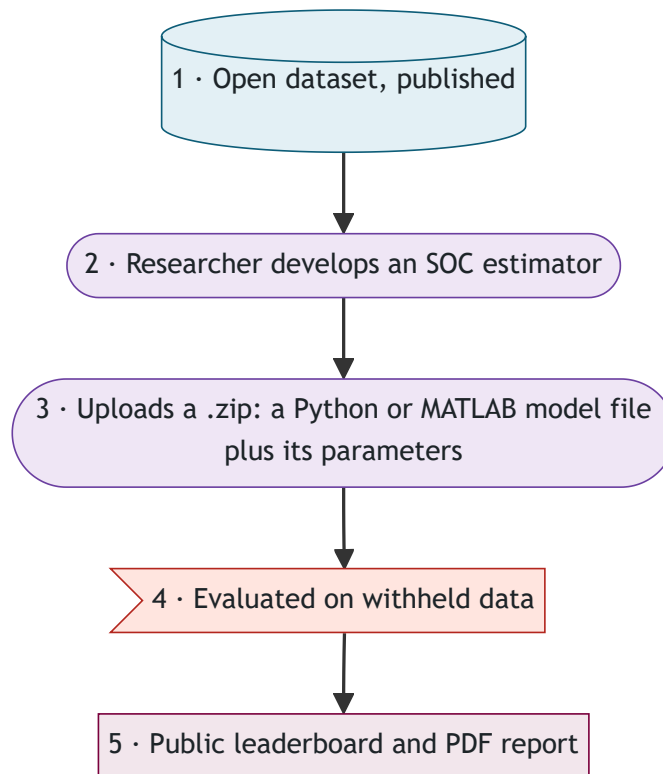


Figure 2.2 The five stages. Stage 1 is public and stage 4 is confidential; the benchmark's validity depends on keeping them apart.

2.2 Three programs and one rule

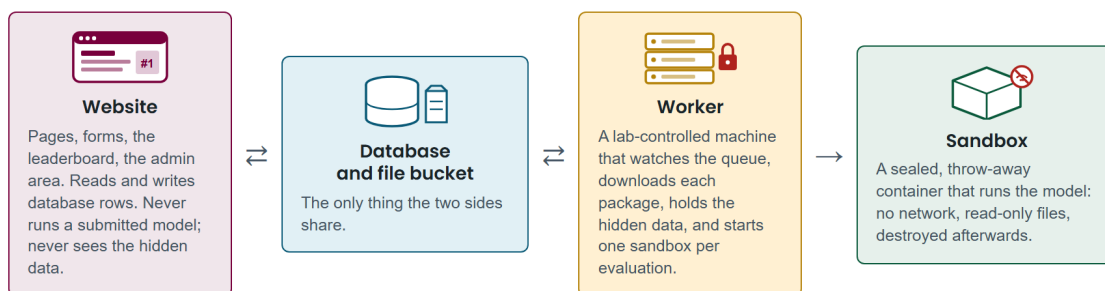
Behind the website are three separate programs, and the architecture follows from one rule:

The web tier never executes submitted code and never has access to the withheld data.

The three programs divide the work as follows:

- **The website** serves pages and forms, records submissions, and shows results. It reads and writes the database and nothing else.
- **The worker** is a separate machine under the laboratory's control. It holds the withheld data, claims queued submissions and runs each evaluation.
- **The sandbox** is an isolated container the worker starts for every evaluation: no network access, a read-only filesystem, discarded when the run ends. It is the only place a submitted model is ever executed.

THE THREE PROGRAMS, AND THE ONE RULE



The website and the worker never talk to each other. Everything passes through the database, so a second worker can be added with no configuration, and the site stays up if the worker goes offline.

Figure 2.3 The three programs. The website and the worker share nothing but the database; the sandbox is the only place a submitted model is ever executed.

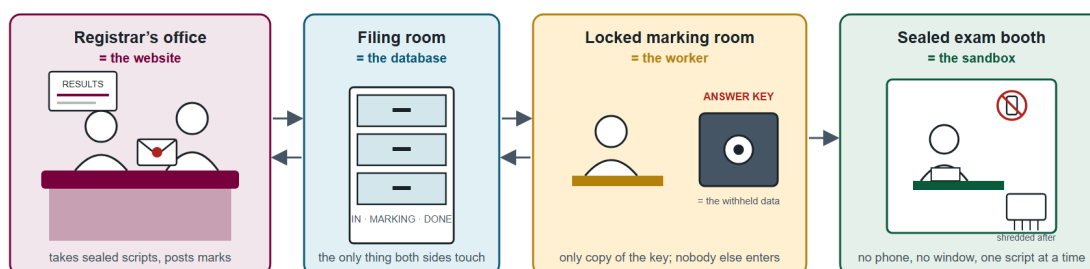
The website and the worker never communicate directly; all coordination passes through the database. Three consequences of this recur throughout the manual:

1. An additional worker can be added with no configuration, since it simply begins claiming jobs.
2. The website remains fully available when no worker is reachable, as it did during a ten-hour network outage in September 2026.
3. The job queue is an ordinary database table rather than a separate message-queue service.

An analogy: the sealed examination

The arrangement is easier to remember as the way a university runs a written examination. Figure 2.4 draws the comparison.

AN ANALOGY: HOW A UNIVERSITY RUNS A SEALED EXAMINATION



The registrar never opens a script and never sees the answer key. The examiner never meets a candidate. Everything passes through the filing room, and every script is destroyed once it has been marked.

Figure 2.4 The platform as a sealed examination. The registrar's office (website) accepts scripts and posts marks; the filing room (database) is the only place both sides touch; the locked marking room (worker) holds the only copy of the answer key (withheld data); each script is marked in a sealed booth (sandbox) and shredded afterwards.

- **The registrar's office is the website.** Candidates enrol, hand in a sealed script, and later read their mark on the noticeboard. The clerks never open a script and have never seen the answer key.
- **The filing room is the database.** The registrar drops each script in the IN tray; the examiner collects it from there and returns the mark to the DONE tray. The two never meet, so a second examiner can start work simply by collecting from the same tray.
- **The locked marking room is the worker.** It holds the only copy of the answer key. Nobody but the examiner enters, and the examiner never leaves with it.
- **The sealed exam booth is the sandbox.** The script is answered against the key one at a time, in a booth with no phone and no window, and is shredded the moment it has been marked. Nothing that happens in the booth can reach the outside, and nothing is kept.

A submitted model is the candidate. It is given the questions (the withheld drive cycles), writes its answers (the estimates), and is marked against the key (the reference state of charge). It never sees the key, and no copy of it survives the examination.

2.3 Where the components are hosted

Each program runs on a different service, chosen so that the platform costs nothing to host and so that the withheld data stays on a machine the laboratory controls. Figure 2.5 shows the four.

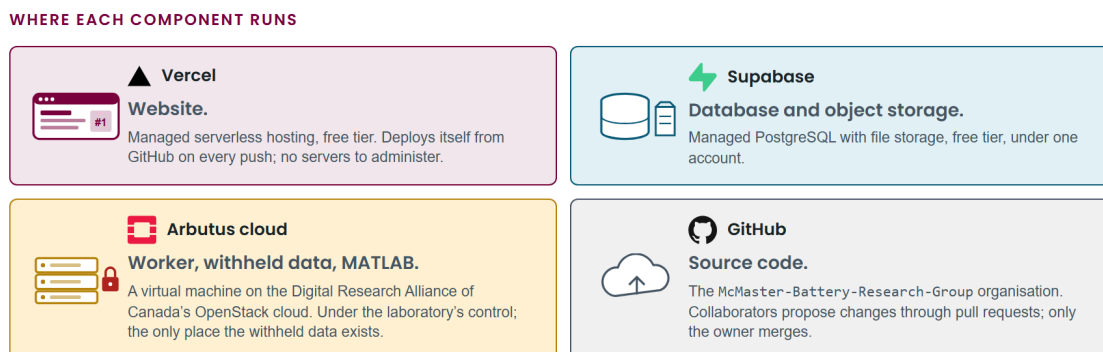


Figure 2.5 Where each component runs. The website on Vercel, the database and file storage on Supabase, the worker on the Alliance's Arbutus cloud, and the source code on GitHub.

2.4 Visual conventions

Every diagram in this manual uses one icon, one colour and one shape for each kind of thing, so a component looks the same on every page. The pictures use the icons; the flow diagrams, which are drawn with a diagramming tool that cannot show icons, use the shapes and colours. Figure 2.6 is the key to both.

HOW TO READ THE DIAGRAMS: ONE ICON, ONE COLOUR AND ONE SHAPE PER KIND OF THING

IN PICTURES	IN FLOW DIAGRAMS		MEANING
		Website	The public site and admin pages. Talks only to the database.
		Worker	The lab machine that runs evaluations and holds the withheld data.
		Sandbox	The sealed container a model runs in. No network; destroyed afterwards.
		Data	The database and file bucket, or a data file.
		External service	Something outside the platform: GitHub, MathWorks, a mail gateway.
		Person	A researcher or administrator.
		Withheld data or risk	The answer key, or a point where something could go wrong.

Figure 2.6 The visual vocabulary of the manual: the icon used in pictures, the shape and colour used in flow diagrams, and what each stands for.

CHAPTER 3

Using the platform

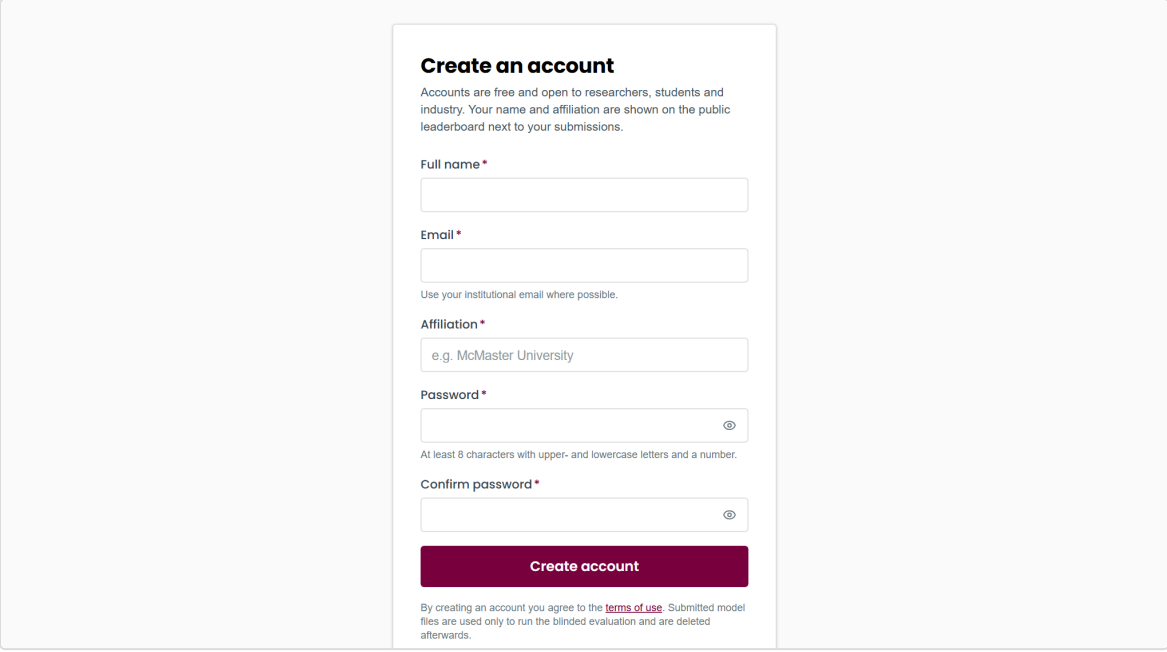
In this chapter. The workflow of a researcher, page by page: registering an account, validating a package before submission, submitting, interpreting the results, and locating the model on the leaderboard.

Files to open: `submit-form.tsx` → `dry-run-panel.tsx` → `submissions/[id]/page.tsx` → `leaderboard-table.tsx`.

3.1 Registration

Registration is a four-step process:

1. The form asks for a name, an institutional affiliation, an e-mail address and a password.
2. A confirmation e-mail follows. Opening its link displays a page with a **Confirm** button.
3. Pressing that button verifies the account. Section 7.1 explains why the link alone is not sufficient.
4. The verified user signs in, and the session remains valid for fourteen days.



Create an account

Accounts are free and open to researchers, students and industry. Your name and affiliation are shown on the public leaderboard next to your submissions.

Full name *

Email *

Use your institutional email where possible.

Affiliation *

e.g. McMaster University

Password *

At least 8 characters with upper- and lowercase letters and a number.

Confirm password *

Create account

By creating an account you agree to the [terms of use](#). Submitted model files are used only to run the blinded evaluation and are deleted afterwards.

Figure 3.1 The registration page. The name and affiliation entered here are displayed beside the user's models on the leaderboard.

3.2 Validating a package before submission

Each account is limited to three submissions per day, so it is worth validating a package before spending one. The top of the submission page contains a **Test your package first** panel. Selecting a zip and pressing **Run test** executes the package through the real evaluator on one *public* drive cycle (cell m80, profile REORDERED1, 25 °C, two hours). The model's console output streams onto the page, and the error on that cycle is reported when it completes. A test run is not recorded anywhere public and does not involve the withheld data. Its purpose is to catch the defects that would otherwise waste a submission:

- a model file that is missing or incorrectly named,
- a parameter file the model attempts to load but the package does not contain,
- a model function that returns a value of the wrong shape or outside the interval 0 to 1.

BLINDED EVALUATION

Submit a model

Upload your submission package. Its layout and file names are checked immediately, queued for evaluation on the blinded dataset, and deleted as soon as the evaluation finishes.

Python packages: evaluator paused
last seen 20 d ago · submissions queue and run automatically, in order, when a machine that can run them is back; you will be e-mailed.

MATLAB packages: evaluator online · idle · new submissions start within seconds

Test your package first

Runs your model through the real evaluator on one **public** drive cycle (m80, REORDERED1, 25 °C, 2 h). Catches format and runtime errors and shows your error and complexity — without using a submission, touching the blinded data, or appearing anywhere. Up to 5 per hour.

Choose a .zip package...

Run test

Package checklist

- A single .zip with files at the top level — no sub-folders.
- Model.m, Model.p or Model.py defining $[Y, z] = \text{Model}(X, z)$. $X = [I, V, T]$, Y is SOC in 0–1.
- Any parameter files the model loads (.mat, .npz, ...). Your name, affiliation and model name come from your account and this form.
- Use **Test your package first** above — it runs your model on a public cycle through the real evaluator and catches format and runtime errors before you spend a submission.

[Full submission format guide →](#)

1. Describe the model

Model name *

What happens to your file

Your package is stored only until the evaluator has run.

Figure 3.2 The submission page. The test panel is placed above the submission form; the checklist on the right specifies the package contents.

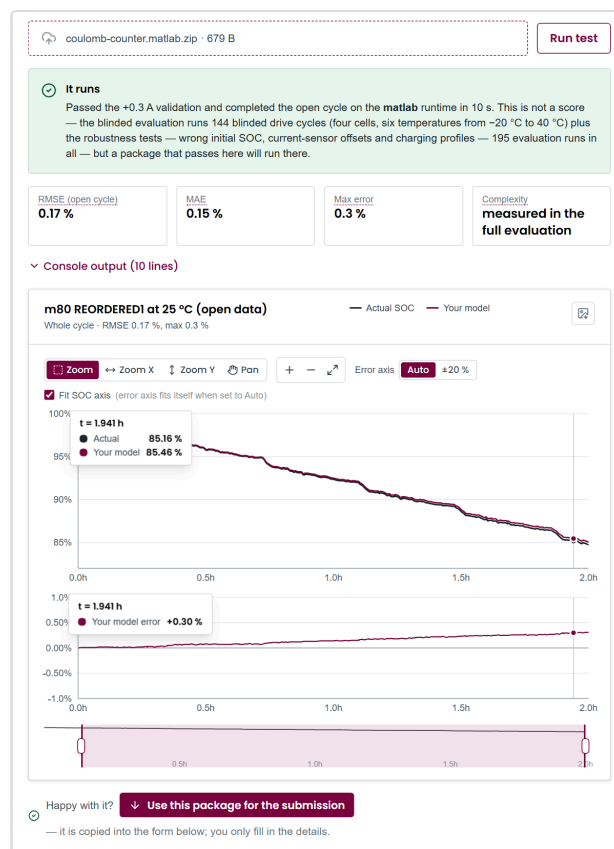


Figure 3.3 A completed test run. The panel reports RMSE, mean absolute error and maximum error on the public cycle, the model's console output, and the estimate plotted against the measured state of charge.

The package format is deliberately minimal:

- All files are placed at the top level of the archive, with no sub-directories.
- Exactly one file is the model: `Model.py` for Python, or `Model.m` or `Model.p` for MATLAB.
- Any parameter files the model loads (`.mat` , `.npz` and similar) are placed alongside it.

Section 5.1 specifies the model function's interface.

3.3 Submission

Below the test panel is the submission form. It collects:

- a model name and a short description,
- the model type (Coulomb counter, extended or unscented Kalman filter, feed-forward, LSTM or GRU network, transformer, physics-based, hybrid, or other),
- whether the model is to be private,
- an optional contest to enter,
- any co-authors.

The form is validated in the browser with the same rules the server applies, so an error is reported before the upload begins. Once a package has been submitted, the page shows its position in the queue, an estimated start time and whether a worker capable of running the package's language is currently online. When evaluation begins, the page switches to a live progress indicator and the model's console output.

3.4 Interpreting results

The results page is organised to present interpretation before detail. It opens with a plain-language reading of the scorecard, generated from the numbers themselves:

- which test case contributes most to the weighted score,
- whether accuracy on the withheld cell matches accuracy on the open cells, which indicates generalisation rather than memorisation,
- how accuracy varies with temperature,
- whether the model recovers from an incorrect initial state of charge,
- whether the model tolerates a biased current measurement.

Each statement can be verified against the scorecard immediately below it.

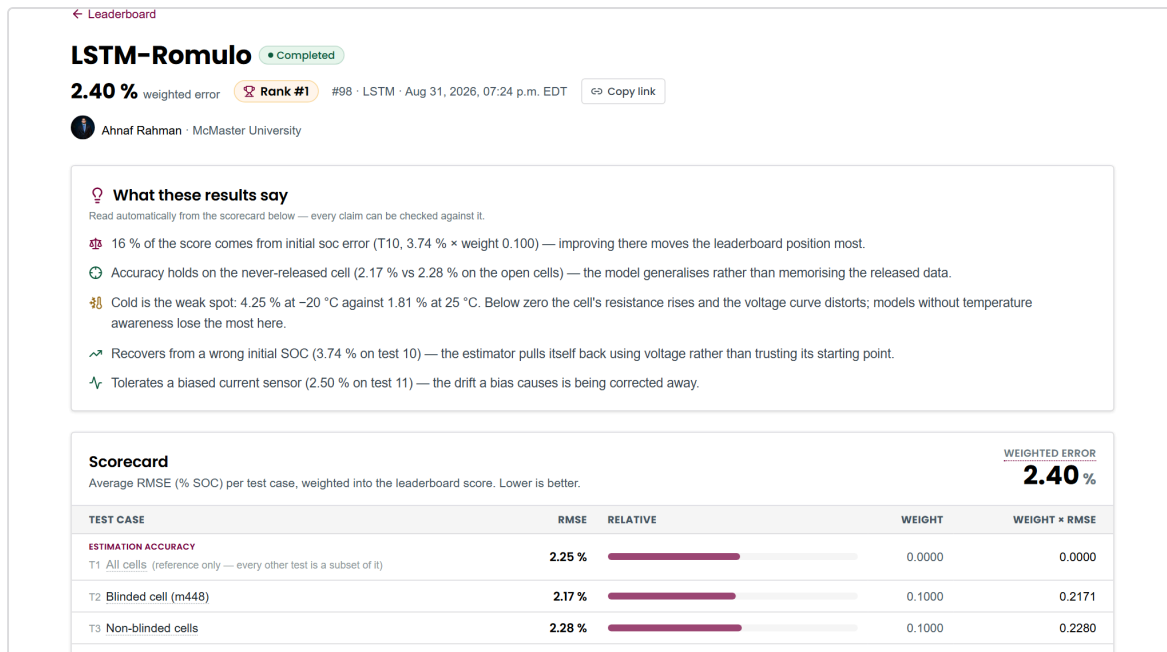


Figure 3.4 The head of a results page. The interpretation is generated from the scorecard beneath it.

The scorecard lists the eighteen test cases with the RMSE, the weight and the product of the two. The products sum to the weighted error shown at the foot of the table, so the derivation of the score is fully visible. Below the scorecard are the key traces: the cycles that most differentiate estimators (the temperature extremes, the withheld cell and the robustness runs), each plotted against the measured state of charge with interactive zoom. The remaining material is available in collapsed sections:

- the per-cycle table covering all 144 cycles,
- the score history, recording every revision of the result,
- the downloads: the PDF report, the results as JSON, and the full-resolution traces.

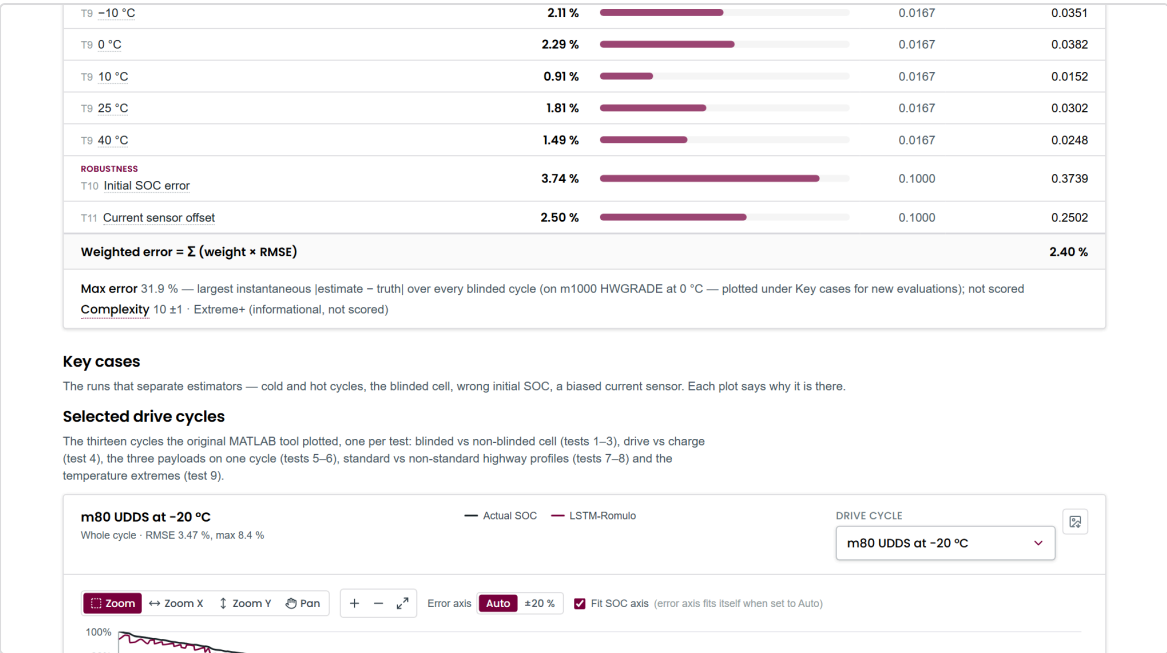


Figure 3.5 The foot of the scorecard and the first key trace. The weighted error is the sum of the weight $\times$ RMSE column.

3.5 The leaderboard

The leaderboard ranks every public, completed model by weighted error in ascending order. Ties are broken by the all-cells error and then by submission time. The table offers:

- filters by author, affiliation and model type,
- additional columns, including the per-temperature and robustness cases, through the **Columns** control,
- export as CSV,
- a link from each model to its results page, and a **Compare** page that overlays two to four models on the same charts.

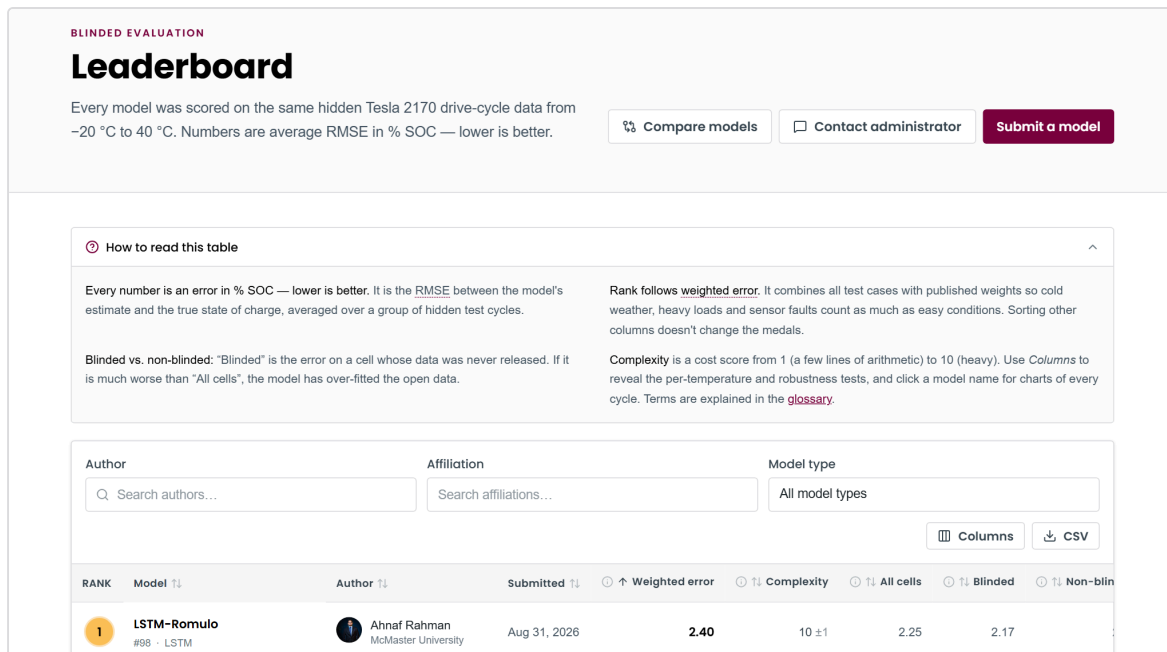


Figure 3.6 The leaderboard, with its explanatory panel expanded.

A signed-in user's private models are shown to that user with a dashed provisional rank, indicating the position the model would occupy, without displacing any public entry. Results produced by an earlier version of the scoring code remain listed but unranked, and their authors are invited to resubmit. Because packages are deleted after evaluation, a re-evaluation cannot be performed automatically.

3.6 Co-authors and contests

A co-author is invited by e-mail and must accept by pressing a button on the invitation page; only then is the co-author listed publicly and included in result notifications. A **contest** is a time-bounded event with its own leaderboard, frozen at the closing date. Contest entries must remain public, and their name and model type are fixed once the contest closes.

COMPETITIONS

SOC estimation contests

Time-boxed challenges with cash prizes, judged on a frozen snapshot of the blinded evaluation at the deadline. Register, submit up to the entry limit, and track your standing on the contest leaderboard.

Open now



PRIZES

CASH\$5,000 first prize ·
CASH\$2,000 second ·
CASH\$1,000 third

2026 Battery SOC Estimation Challenge Open

Build the most accurate and robust state-of-charge estimator for the Tesla 2170 cell across -20°C to 40°C . Cash prizes for the top three weighted-error scores.

📅 Aug 11, 2026 → **Oct 9, 2026** 👤 1 registered · 0 entries

[View contest & register](#)

 **BatterySOCBenchmark**

Benchmark
Leaderboard

Learn
Get started

About
About the project

Cite
P. J. Kollmeier, M. Naouib, F.

Figure 3.7 A contest page: rules, timeline and the contest leaderboard.

CHAPTER 4

The life of a submission

In this chapter. The path of a package from the browser to the leaderboard: validation, queueing, atomic claiming by a worker, sandboxed execution, storage and reporting, and the handling of failure.

Files to open: `submit/actions.ts` → `package-check.ts` → `run-job.ts` → `python-evaluator.ts`.

4.1 Overview

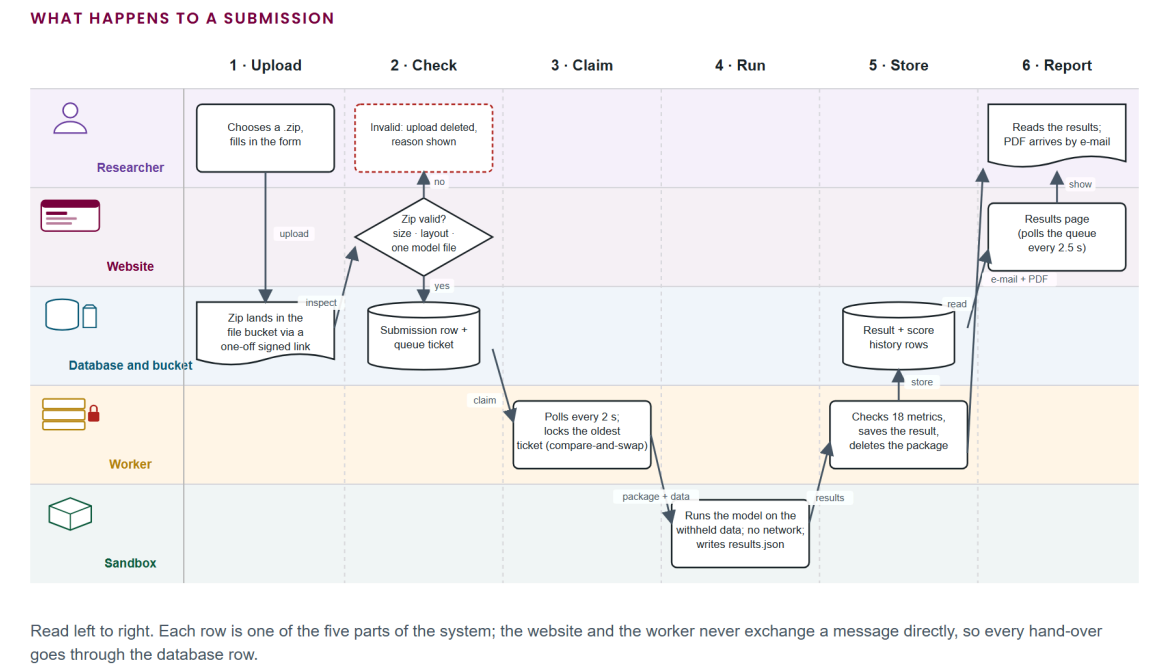


Figure 4.1 The six stages as a swim-lane flowchart. Each row is one part of the system; a box is an action, a diamond a decision, a cylinder a database write, a wavy-bottomed box a document. Every hand-over between the website and the worker passes through the database row.

4.2 The stages in detail

Stage 1 · Upload

Most web hosts limit the size of a single request, and Vercel's limit is 4.5 MB. A submission package can be ten times that, so the archive cannot travel through the website at all. Instead the browser asks the website for permission, receives a **signed URL** (a storage address that authorises exactly one upload and then expires), and sends the file straight to the bucket. Figure 4.2 shows the four exchanges.

STAGE 1 — THE UPLOAD GOES AROUND THE WEBSITE, NOT THROUGH IT



Figure 4.2 The upload goes around the website. The browser obtains a single-use signed URL, sends the archive directly to the file bucket, and submits the form with nothing but the storage key.

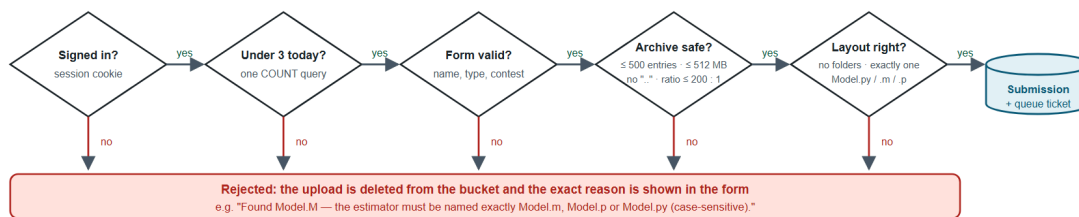
1. The browser calls `POST /api/upload` with the file's name and size. The route refuses anything that is not a `.zip`, anything over the configured maximum, and any caller who is not signed in.
2. The website asks the storage service for a signed upload URL and returns it together with the **object key**, the path the file will have in the bucket (for example `submissions/7f3a...zip`).
3. The browser sends the archive to that URL with an HTTP `PUT`. The website is not involved; the transfer is between the browser and the bucket.
4. The submission form is posted with the object key in a hidden field. The form itself is a few hundred bytes.

The same route serves the test-run panel, with a different prefix so that test uploads and real submissions never share a folder.

Stage 2 · Validation

The form is handled by one **server action**, a function that runs on the web server when the form is posted. It performs every check in order of cost, so that the expensive step of fetching and unpacking the archive is reached only by requests that have already passed the free ones. Figure 4.3 lays the gate out.

STAGE 2 — THE VALIDATION GATE, CHEAPEST CHECK FIRST



Checks run in this order so that the expensive one (fetching and unpacking the archive) is only reached by requests that have already passed the free ones.

Figure 4.3 The validation gate. Each diamond is a check; the first to fail stops the process, deletes the upload and returns the reason to the form.

1. **Signed in?** The session cookie is verified. An anonymous request is redirected to the sign-in page.
2. **Under the daily limit?** One database count of the user's submissions in the last 24 hours. Three is the default; administrators are exempt.
3. **Form valid?** The model name, description, type, privacy flag, contest and co-author addresses are checked against the same rules the browser applied.
4. **Archive safe?** The archive is fetched from the bucket and its table of contents inspected without unpacking. It is rejected if it has more than 500 entries, would exceed 512 MB unpacked, contains any entry over 256 MB, contains a path with `..` or an absolute path, or has any entry whose compression ratio exceeds 200 to 1 (the signature of a decompression bomb).
5. **Layout right?** There must be no sub-directories, and exactly one of `Model.py`, `Model.m` or `Model.p` at the top level. A MATLAB file is additionally checked for the required function signature.

The check code is deliberately specific about what it found, because a rejected submission should not need a support request:

```

if (n.startsWith("/") || /^[A-Za-z]:/.test(n) || n.split("/").includes(".."))
  problems.push(`Unsafe path in archive: "${e.entryName}".`);
if (size / comp > MAX_RATIO && size > 1024 * 1024)
  problems.push(`"${e.entryName}" has an implausible compression ratio; the p
  
```

On any failure the uploaded object is deleted from the bucket and the form is redisplayed with the message. On success the `Submission` row and its `EvaluationJob` are created in one write, with the package's language recorded so that only a worker capable of running it will claim it.

Stage 3 · Queueing

The researcher is redirected to the submission's page, which polls `GET /api/submissions/{id}/status` every 2.5 seconds. The response is assembled from two tables and nothing else:

- the **job queue**, which gives the submission's position among unclaimed jobs of the same language and an estimated wait derived from recent evaluation times;
- the **worker heartbeats**, one row per worker, refreshed every fifteen seconds with the languages it supports, what it is running, and its last console lines. A worker whose heartbeat is older than a minute is shown as offline.

```
{ "status": "QUEUED", "runtime": "matlab", "queuePosition": 2, "queueWaitSec": 10,
  "evaluator": { "online": true, "queued": 2, "running": 1, "capacity": 1 } }
```

If no online worker supports the package's language, the page says so rather than showing an estimate that cannot be met.

Stage 4 · Claiming

Every worker polls the queue every two seconds. Claiming has to be safe when two workers poll at the same instant, and the platform uses no locking service; the database's own atomic update is enough. The worker reads the oldest unclaimed job for its language, then issues one conditional update: set the lock, but only if the row's lock timestamp is still what was just read. Either exactly one row changes, and the job is this worker's, or zero rows change, and another worker got there first.

```
const { count } = await db.evaluationJob.updateMany({
  where: { id: candidate.id, lockedAt: candidate.lockedAt }, // unchanged s
  data: { lockedAt: new Date(), lockedBy: workerId(), attempts: { increment: 1 } },
});
return count === 1 ? candidate : null; // 1 = ours,
```

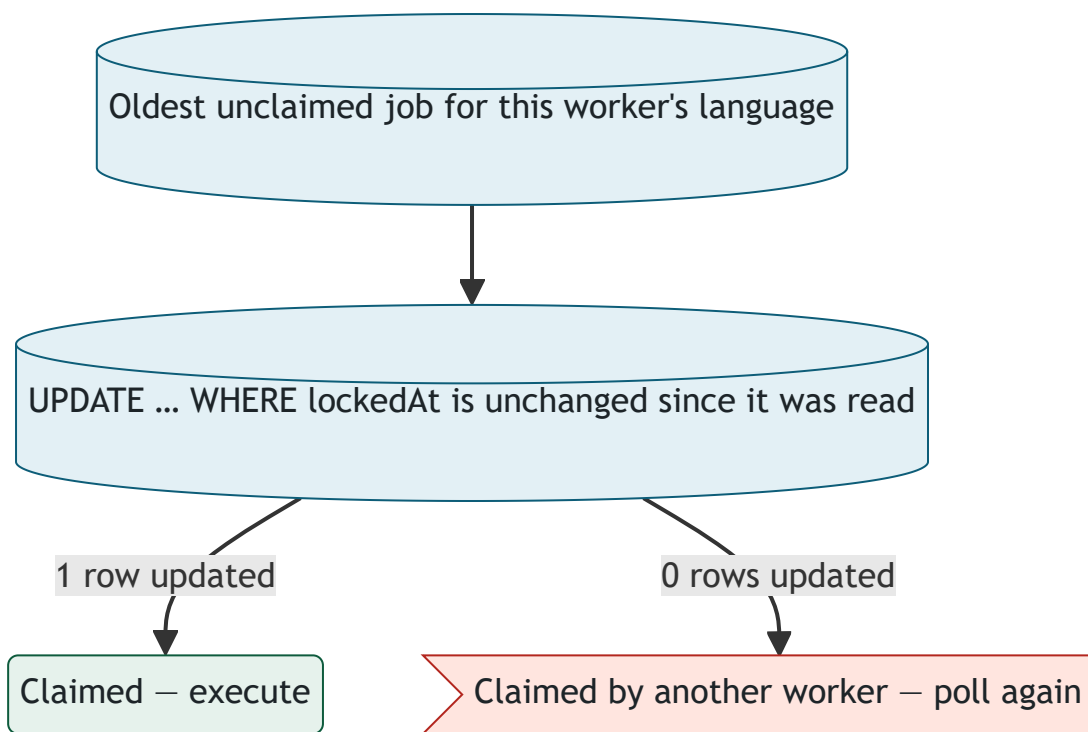


Figure 4.4 Claiming a job by compare-and-swap. One attempt has exactly two outcomes.

Two further rules keep the queue honest:

- Every progress line the model prints refreshes the lock timestamp, so a job that is genuinely running is never mistaken for an abandoned one, however long it takes.
- A lock older than thirty minutes with no progress is treated as abandoned (the worker crashed or lost power) and the job becomes claimable again. Each job is allowed two attempts in total.

Stage 5 • Execution

The worker unpacks the package into a scratch directory and starts one Docker container for the run. Figure 4.5 shows exactly what the container is given and what it is denied.

STAGE 5 — WHAT THE SANDBOX CAN SEE, AND WHAT IT CANNOT

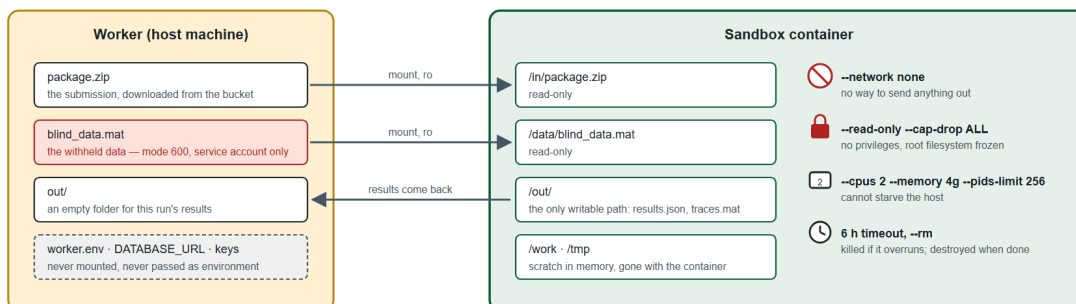


Figure 4.5 The sandbox's view of the world. Three paths are mounted from the host; everything else the worker holds, including its credentials, is invisible. The flags on the right are what make the container a sandbox rather than merely a container.

The `docker run` invocation the worker issues, reduced to its essentials:

```
docker run --rm --name eval-7f3a \
  --network none \
  --read-only --tmpfs /work:rw,exec,size=2g --tmpfs /tmp:rw,size=512m \
  --cap-drop ALL --security-opt no-new-privileges \
  --user 998:998 --pids-limit 256 --memory 4g --cpus 2 \
  -v /var/lib/socbench/jobs/7f3a/package.zip:/in/package.zip:ro \
  -v /var/lib/socbench/jobs/7f3a/out:/out:rw \
  -v /var/lib/socbench/blind-data/blind_data.mat:/data/blind_data.mat:ro \
  socbench-eval /in/package.zip /out --data /data/blind_data.mat
```

Reading the flags in order:

- `--network none` removes the network interface. The model can read the withheld data, as it must, but has no way to transmit anything.
- `--read-only` with two in-memory `--tmpfs` scratch areas means the container's filesystem cannot be modified except in places that vanish with it.
- `--cap-drop ALL` and `no-new-privileges` remove every Linux capability and forbid escalation, and `--user` runs the process as the worker's own unprivileged account.
- The CPU, memory and process-count limits stop a runaway model from starving the host or the other evaluation.
- The three `-v` mounts are the whole of the container's contact with the host. The worker's environment, which holds the database and mail credentials, is never passed in. For MATLAB packages the worker adds a single 24-hour licence token and nothing else.

Everything the container writes to its console is appended to the job's log row, which is what the submission page shows as the live console and what refreshes the lock. A timeout (six hours by default, changeable by an administrator) and the owner's **Cancel** button both stop the container, and stopping it also ends any MATLAB process inside.

Stage 6 · Storage and reporting

When the container exits, the worker reads `results.json` from the output directory and proceeds through a fixed sequence. The order matters: the package is deleted only after the result is safely stored, and the e-mail is sent only after the package is gone.

1. **Verify.** All eighteen metrics must be finite numbers, and the weighted score is recomputed from the active weights as a cross-check against the evaluator's own figure.
2. **Store the traces.** The full-resolution traces file (about 6 MB) goes to the bucket, replacing the previous one if this is a re-evaluation.
3. **Commit.** The result row is written (or updated) and the submission is marked complete in a single **transaction**, a group of database writes that either all succeed or all fail. A partial result can never be visible.
4. **Record history.** A `ScoreRevision` entry is appended with the score, the metrics and the evaluator version, so the result can be reconstructed after any later re-scoring.
5. **Delete the package.** The uploaded archive is removed from the bucket. From this point no copy of the submitted model exists anywhere in the platform.
6. **Report.** The PDF is generated and e-mailed to the owner and every confirmed co-author.

The line the worker appends to the job log when it is done is the one to look for when watching a job from the workers page:

```
completed – weighted error 2.401 %
```

4.3 Timing

HOW LONG EACH STAGE TAKES — A MID-WEIGHT MODEL, END TO END

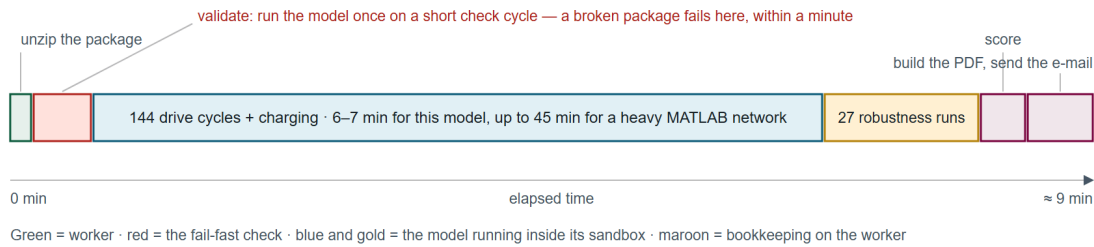


Figure 4.6 The distribution of time across the stages for a typical model. The validation step exists so that a defective package fails within seconds rather than after forty minutes.

4.4 States and transitions

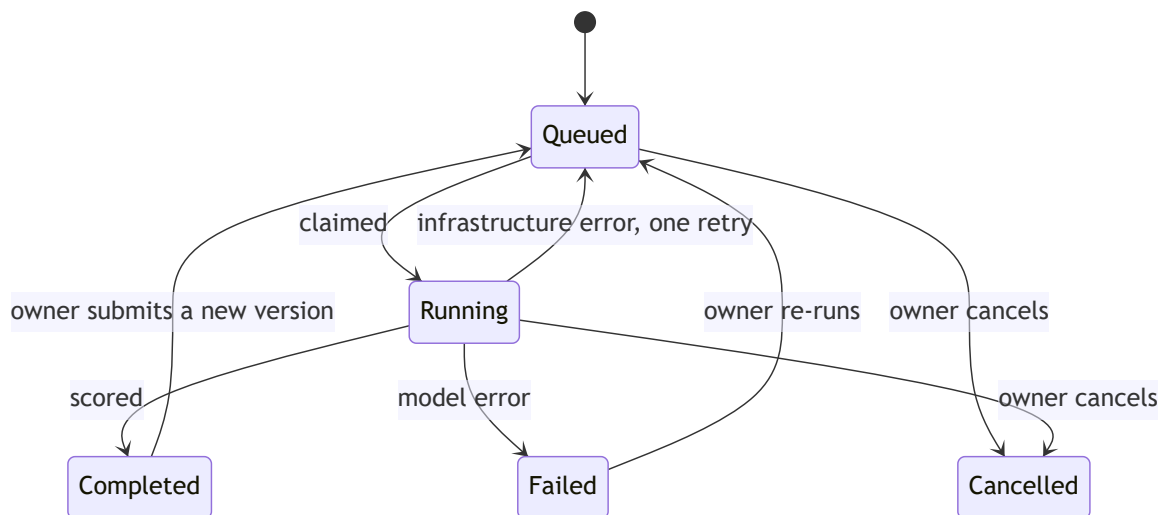


Figure 4.7 Every state a submission can occupy, and the events that move it between them.

A failure attributable to the package (an exception, an invalid return value, or a timeout) is final, and the error message is stored verbatim for the author. A failure attributable to the infrastructure (the container runtime unavailable, or a defect in the worker) is retried once. Stopping a worker returns its current job to the queue without consuming an attempt.

CHAPTER 5

Scoring methodology

In this chapter. The model interface, the composition of the evaluation set, the aggregation of per-cycle errors into eighteen test cases and one weighted score, and the two auxiliary quantities reported alongside it.

Files to open: `pipeline.py` (`score` , `complexity`) $\rightarrow$ `runner.py` $\rightarrow$ `Run_Model.m` $\rightarrow$ `test-cases.ts`.

The evaluator is approximately 800 lines of Python. It reimplements the laboratory's original MATLAB blind-modelling tool and reproduces its results to three decimal places on the reference models.

5.1 The model interface

A model is a single function invoked once per sample, in the same manner as an estimator running on a vehicle's battery controller. At each call it receives the current, voltage and temperature for that sample together with whatever state it returned from the previous call, and it returns its estimate and the state to carry forward. It has no access to future samples. The simplest admissible model is a Coulomb counter, which integrates the measured current over the rated capacity Q (here 4.6 Ah) at the one-second sample interval Δt :

$$\widehat{\text{SOC}}_k = \widehat{\text{SOC}}_{k-1} + \frac{I_k \Delta t}{3600 Q}, \quad \widehat{\text{SOC}}_0 = 1 \quad (5.1)$$

In code it is four lines:

```
def Model(X, z=None):          # X = [current, voltage, temperature]
    current = float(X[0])
    soc = 1.0 if z is None else float(z) + current / 3600 / 4.6
    return soc, soc            # (estimate in 0..1, state for the next call)
```

The two supported languages are executed differently but scored identically:

- **Python** models are imported and iterated in-process.
- **MATLAB** models are executed through a single `matlab -batch` session running a forty-line driver script whose only function is the same iteration.

Both paths produce identical scores, verified against the four reference models.

5.2 The evaluation set

WHAT EVERY MODEL IS TESTED ON

Cell (payload it was driven with)	-20 °C	-10 °C	0 °C	10 °C	25 °C	40 °C
m80 — 80 kg, one passenger	6 cycles	6 cycles	6 cycles	6 cycles	6 cycles	6 cycles
m448 — 448 kg, HVAC on · never published	6 cycles	6 cycles	6 cycles	6 cycles	6 cycles	6 cycles
m448N — 448 kg, HVAC off	6 cycles	6 cycles	6 cycles	6 cycles	6 cycles	6 cycles
m1000 — 1000 kg, towing a trailer	6 cycles	6 cycles	6 cycles	6 cycles	6 cycles	6 cycles

4 cells × 6 temperatures × 6 drive cycles = 144 test cycles, plus charging cycles

The 6 cycles at each temperature: UDDS, HWFET, LA92, US06 (standard) · HWCUST, HWGRADE (custom, never published)

+ 9 robustness runs

the model started believing the battery is at 90 %, 60 % or 30 % when it is really full

+ 18 robustness runs

the current sensor reads ±0.05, ±0.1 or ±0.3 A wrong the whole time

Figure 5.1 The evaluation grid. Four cells, six temperatures and six drive cycles give 144 test cycles; the m448 row is the withheld cell.

Before each cycle, one hour of its first sample is prepended as **padding**, so that filters and recurrent networks (models that carry state from one sample to the next) reach steady state before the scored portion begins; the padding is excluded from all metrics. A short **validation run** on one cycle precedes the full evaluation, so that a defective model fails within seconds rather than after forty-five minutes. In total an evaluation comprises 195 runs: the 144 drive cycles, the charging profiles, and 27 robustness runs.

5.3 From per-cycle errors to one score

FROM 144 ERROR NUMBERS TO ONE SCORE

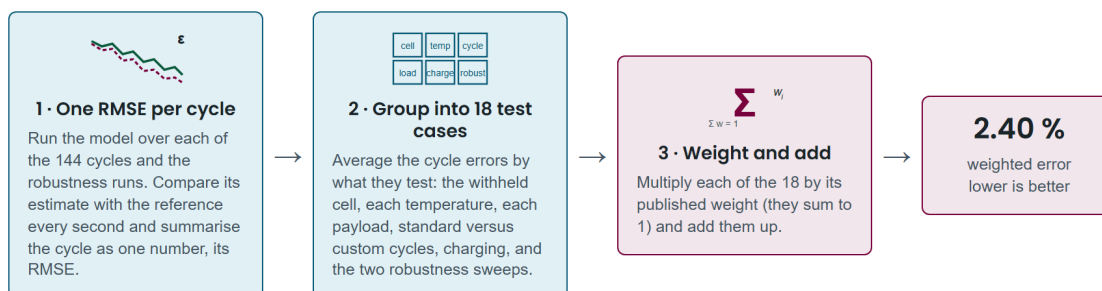


Figure 5.2 The scoring pipeline. Each drive cycle yields one RMSE (ϵ); the RMSEs are grouped into eighteen test cases; the test cases are weighted and summed (Σ).

The leaderboard score is the weighted sum of the eighteen test-case errors, where RMSE_i is the mean RMSE of the cycles in test case i and w_i its published weight:

$$E = \sum_{i=1}^{18} w_i \text{RMSE}_i, \quad \sum_{i=1}^{18} w_i = 1 \quad (5.2)$$

The weights are fixed and published:

- Seven categories carry a weight of 0.1 each: the withheld cell, the open cells, charging, standard cycles, non-standard cycles, the initial-SOC perturbation and the current-offset perturbation.
- The four payload conditions together carry 0.2.
- The six temperatures together carry 0.1.
- The all-cells case is reported on every scorecard but carries a weight of zero, since every other case is a subset of it and a non-zero weight would count each cycle twice.

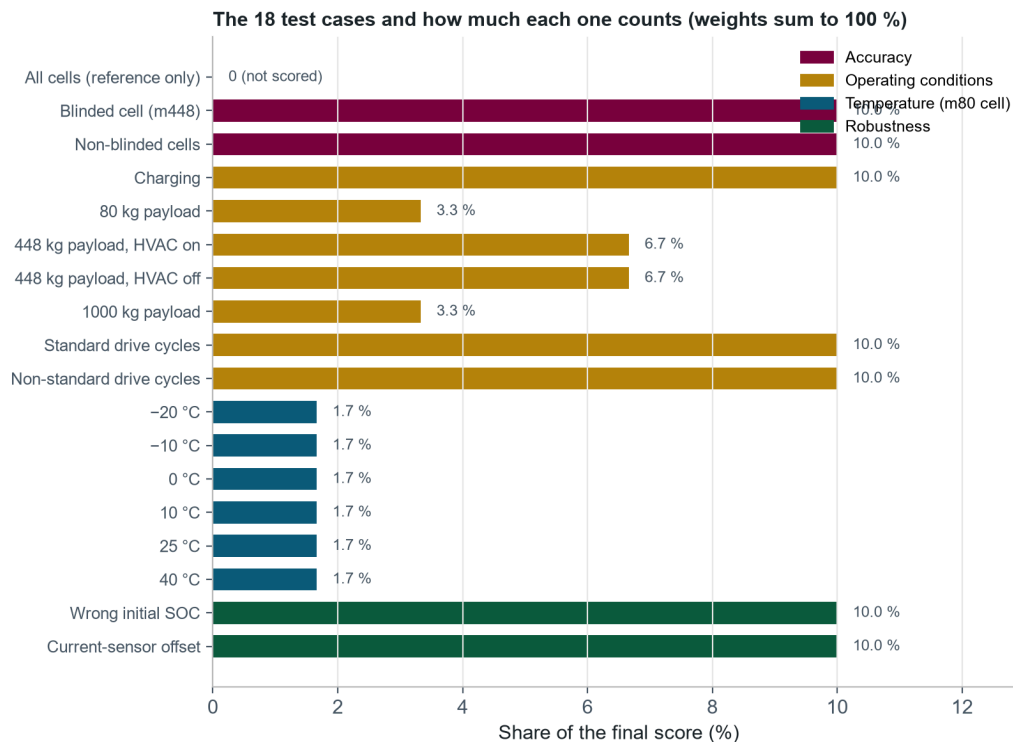


Figure 5.3 The eighteen test cases and their weights. Robustness and generalisation are weighted as heavily as raw accuracy.

5.4 Auxiliary quantities

Complexity is reported on a scale from 1 to 10 and addresses the question of whether a model could run on an embedded battery controller. It is the model's execution time per sample relative to a Coulomb counter measured on the same machine in the same language. It is displayed on the leaderboard but has no effect on ranking.

A **suspicious** flag is raised when the mean error exceeds 25 %. The original tool suppressed such results; this implementation records them for review by an administrator.

5.5 Outputs

Four artefacts leave the sandbox and are written to the database and object storage:

- the eighteen metrics and the weighted score,
- one record per cycle with its RMSE, mean absolute error and maximum error,
- down-sampled traces of thirteen representative cycles, the robustness runs and the model's worst cycle, down-sampled by a method that preserves every extremum so that the plotted trace always agrees with the reported maximum error,
- a 6 MB `.mat` file containing every run at full resolution.

CHAPTER 6

The data model

In this chapter. The tables on which every other component operates, the two conventions that keep them consistent, and the single schema file that defines them.

Files to open: `prisma/schema.prisma`. A single top-to-bottom reading is the most efficient introduction to the system.

Every operation described in Chapters 4 and 5 reads or writes rows in this database. A user owns submissions. Each submission has exactly one queue entry, at most one current result, and an append-only history of every change made to it. Further tables hold contests, worker heartbeats, evaluation settings and the administrative activity log. One file, the Prisma schema, defines all of them, and both the website and the worker are generated from it.

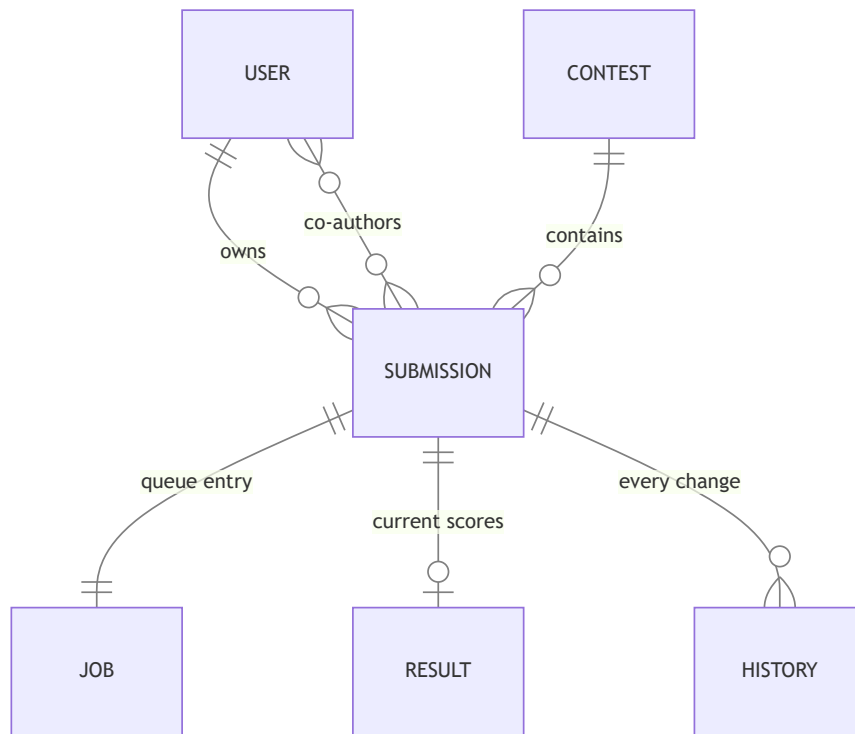


Figure 6.1 The core tables. Each line reads as "has": a user has many submissions; a submission has exactly one job and at most one result.

Two conventions apply throughout:

- **Cascading deletion.** Deleting a user removes that user's submissions, jobs, results and history, so no orphaned rows can exist.
- **Append-only history.** Score revisions, weight changes and administrative events are never modified after they are written, so any past state can be reconstructed.

Several tables stand outside the diagram:

- `WorkerHeartbeat` : one row per worker, refreshed every fifteen seconds with its load, supported languages and most recent console lines.
- `EvalSettings` : a single row of timeouts and the daily submission limit, editable by administrators.
- `ScoringConfig` : weight overrides; the most recent row is authoritative.
- `AdminEvent` : the administrative activity log.
- `RateLimitHit` : counters supporting the rate limits of Chapter 7.
- `ContactMessage` : messages received through the contact form.

Column-level detail is given in the appendix.

CHAPTER 7

Accounts and security

In this chapter. Authentication and session handling, the reason e-mail verification requires an explicit action, and the layered controls that contain a hostile submission.

Files to open: `auth.ts` → `middleware.ts` → `(auth)/actions.ts` → `rate-limit.ts` → `python-evaluator.ts`.

7.1 Why verification requires a button

Corporate mail-security gateways follow every link in an incoming message before the recipient opens it. When the verification link itself performed the verification, these gateways consumed the single-use token and the recipient found it already invalid. The current design separates the two steps:

1. The link in the e-mail leads to a page and changes nothing.
2. The account is verified only when the person presses the **Confirm** button on that page.

In Figure 7.1 the gateway's visit is the third message; the account's state does not change until the person acts.

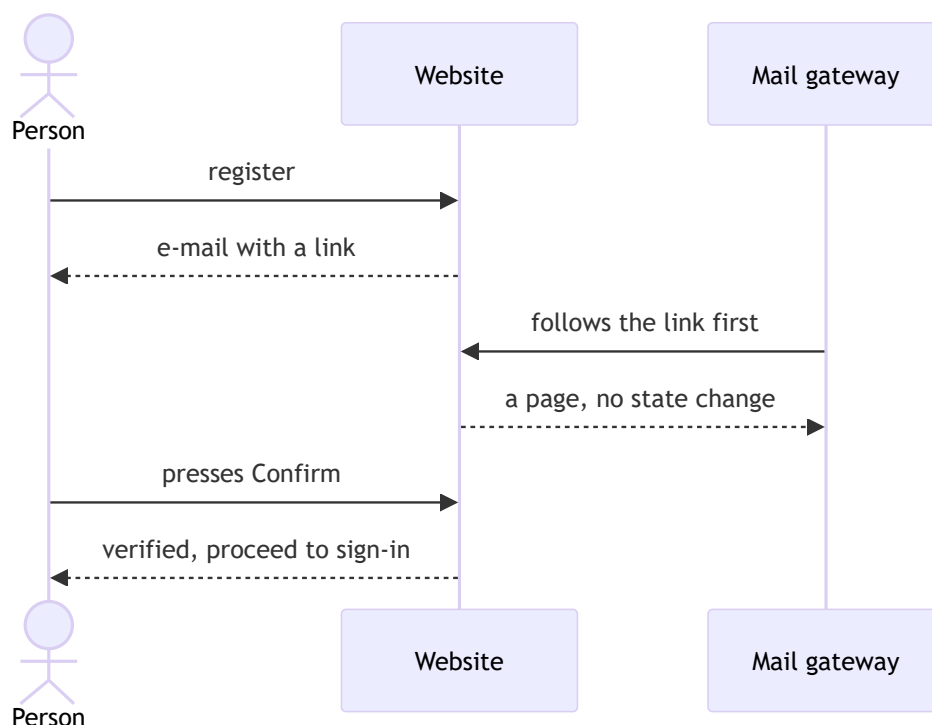


Figure 7.1 Verification is robust to mail gateways because following the link has no side effect.

7.2 Authentication and sessions

Sign-in follows four rules:

1. **Attempt limits.** An account may attempt sign-in ten times per fifteen minutes, and a network address forty times, after which further attempts are refused for the remainder of the window.
2. **Password storage.** Passwords are stored as **bcrypt** hashes, a deliberately slow one-way function: a single verification is imperceptible, but an exhaustive guessing attack is impractical.
3. **Sessions.** A successful sign-in issues a signed **session token** held in a browser cookie and valid for fourteen days. Requests are authenticated by verifying the signature, without a database lookup.
4. **Authorisation.** Routes under `/submit`, `/profile` and `/admin` redirect unauthenticated visitors before rendering, but this is a convenience only. The authoritative check is repeated inside every server action.

7.3 Containing a hostile submission

Secrets are held in exactly three locations: Vercel's environment configuration, a file on the virtual machine readable only by the worker's service account, and a GitHub repository secret. They appear in no repository file, container image, log or e-mail. The remaining threats, and the control that addresses each, are set out in the table below.

THREAT	CONTROL
The model exfiltrates the withheld data	The container has read access (it must) but no network interface, and is destroyed after the run
The model attacks the host	Read-only filesystem, no capabilities, CPU and memory limits, execution as an unprivileged user
The model reads the worker's secrets	The container does not inherit the worker's environment; MATLAB receives only a 24-hour licence token
A decompression bomb or a path-traversal entry	Limits on entry count, unpacked size and compression ratio; no sub-directories; no <code>..</code> components; checked in the web tier and again in the evaluator
The model never terminates	A timeout enforced both inside and outside the container
Queue flooding	Three submissions per day and five test runs per hour per account
Password guessing	bcrypt hashing and the attempt limits above
Account enumeration	Password-reset and resend-verification requests respond identically whether or not the address exists
A compromised administrator removes the others	Administrators cannot be deleted until demoted, and no user can change their own role

CHAPTER 8

Administration and monitoring

In this chapter. The administrative interface, the procedure for revising scoring weights without re-evaluating any model, the notification model, and the automated check that detects an unreachable worker.

Files to open: `admin/actions.ts` → `admin-notify.ts` → `worker-health/route.ts`.

8.1 The administrative pages

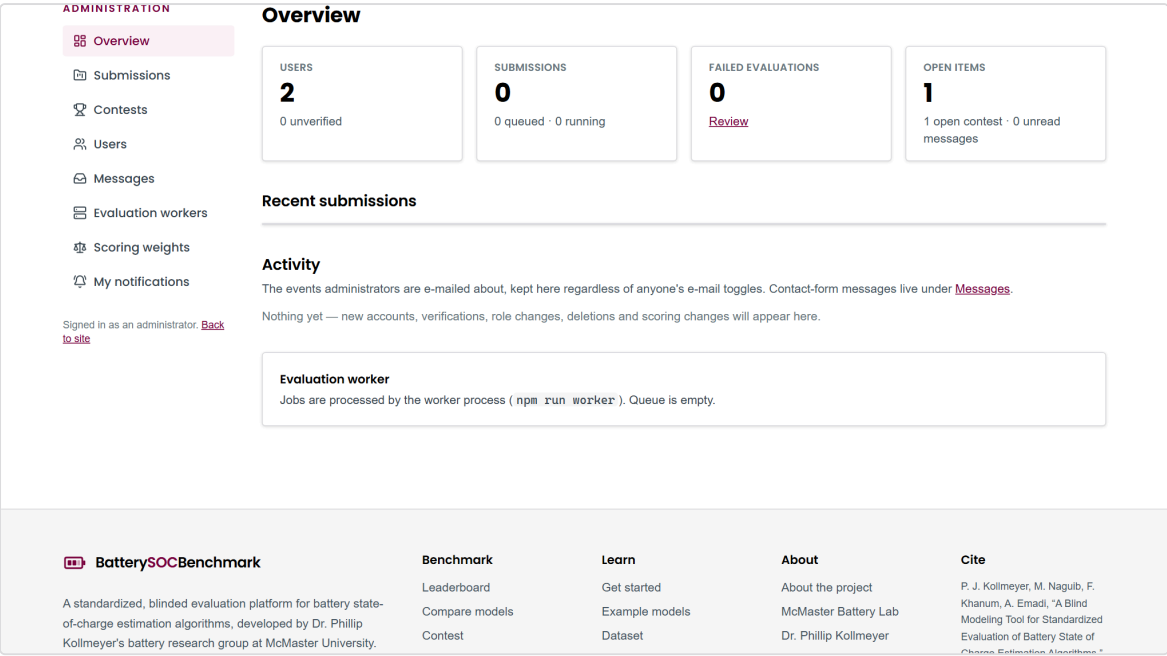


Figure 8.1 The administrative overview: summary counts, recent submissions, the activity log, and the worker's status.

The left-hand navigation enumerates the interface:

- **Overview:** summary counts, recent submissions and the activity log.
- **Submissions:** moderation, retry and bulk deletion.
- **Users:** verification, role assignment and deletion.
- **Contests:** creation and closure.
- **Messages:** the contact-form inbox.
- **Evaluation workers:** every registered machine, the queue, and the evaluation limits.
- **Scoring weights:** the subject of Section 8.2.
- **My notifications:** each administrator's own e-mail preferences.

Every administrative action re-verifies the caller's role before proceeding, and each is subject to a safeguard. The safeguards are tabulated in the appendix.

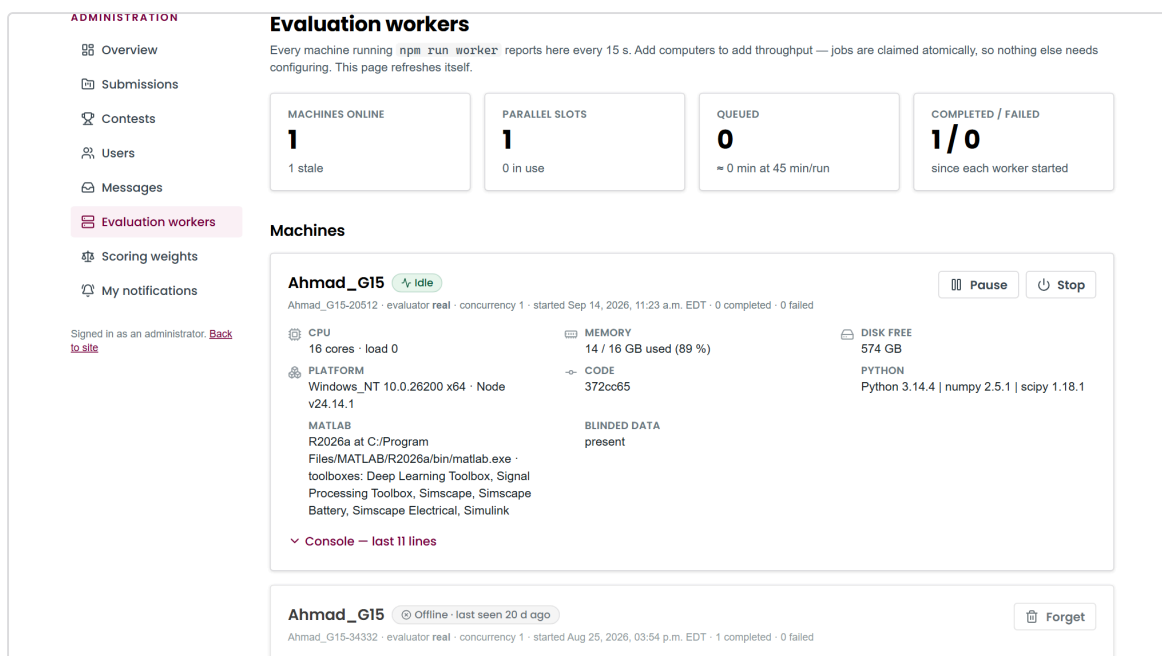


Figure 8.2 The workers page. Each machine reports its supported languages, load, code revision, MATLAB toolboxes, and whether the withheld data is present.

8.2 Revising the weights

An administrator edits the eighteen weights, which must sum to one, previews how many stored scores would change, and saves with a written justification. Every stored result is then re-scored from its persisted per-test metrics; no model is re-executed, since the metrics were retained and the packages were deleted. Authors may optionally be notified of the previous and revised scores with a regenerated PDF. The superseded weights remain in the history.

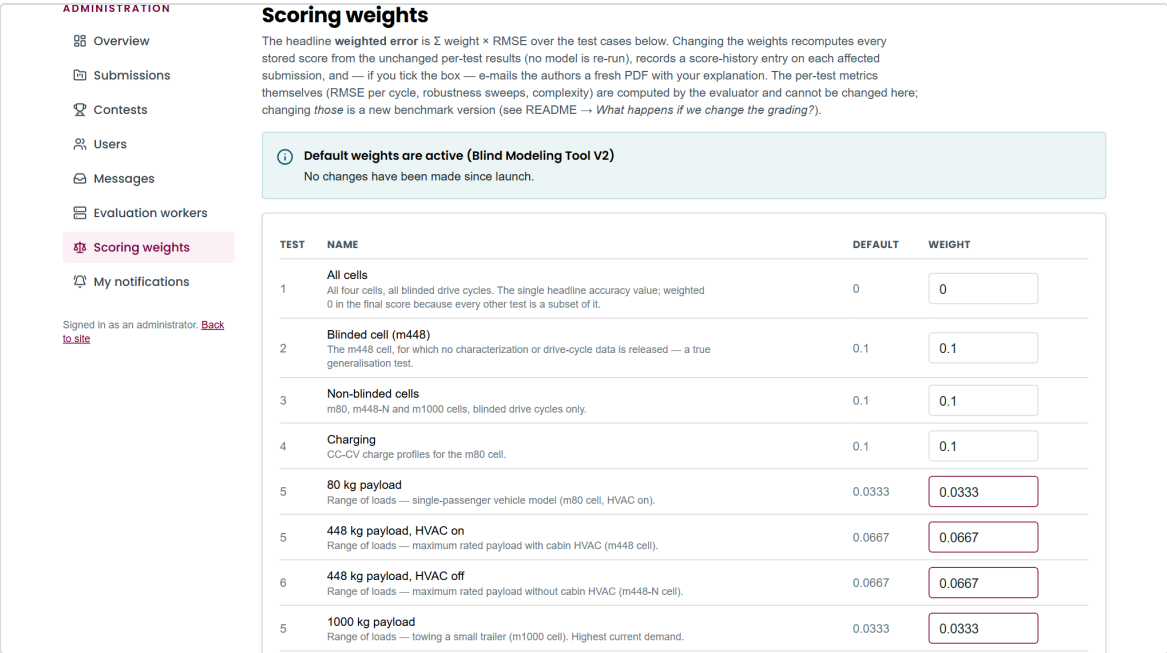


Figure 8.3 The scoring-weights page. Each row is one of the eighteen test cases with its default and current weight.

8.3 Notifications

Every administrative event follows two paths, one unconditional and one configurable:

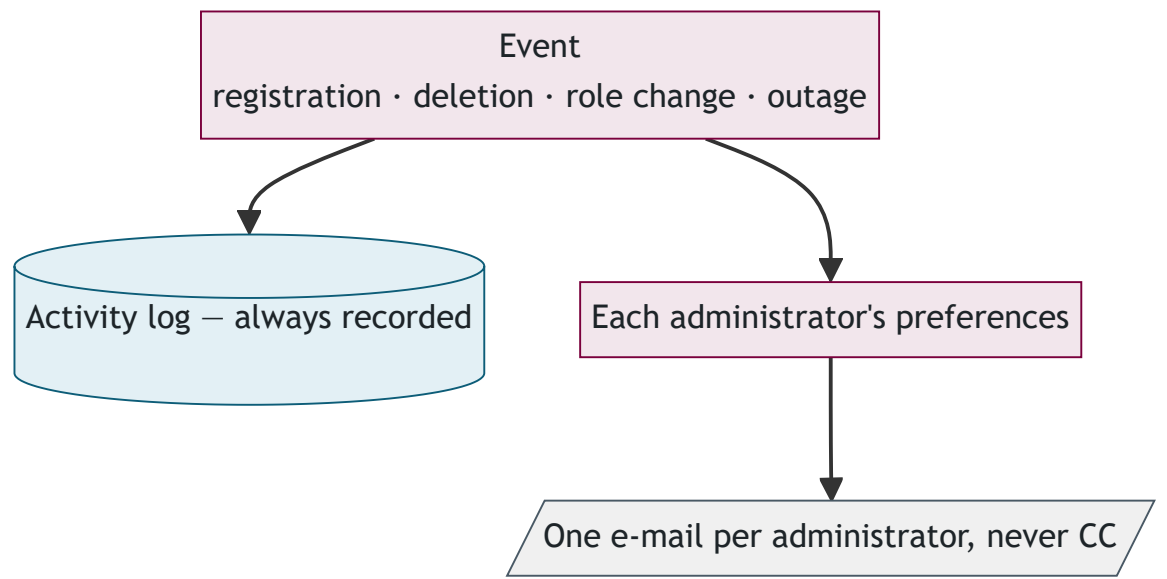


Figure 8.4 The activity log is the record; e-mail is a copy that each administrator enables per category.

One property of the hosting platform deserves emphasis, because it was discovered in production:

- A serverless request is frozen the instant a response is returned.
- Any e-mail dispatched asynchronously without being awaited is therefore silently lost.
- Every such dispatch is wrapped in `after()`, which keeps the request alive until the send completes.

8.4 The outage monitor

The monitor was introduced after an incident in which the worker remained healthy but was unable to reach the database for ten hours, with no indication to anyone. The web tier is the one component that can always observe both the database and the mail service, so it performs the check. A GitHub Actions workflow calls a health endpoint every ten minutes; the endpoint evaluates two conditions and sends an e-mail only when the answer changes.

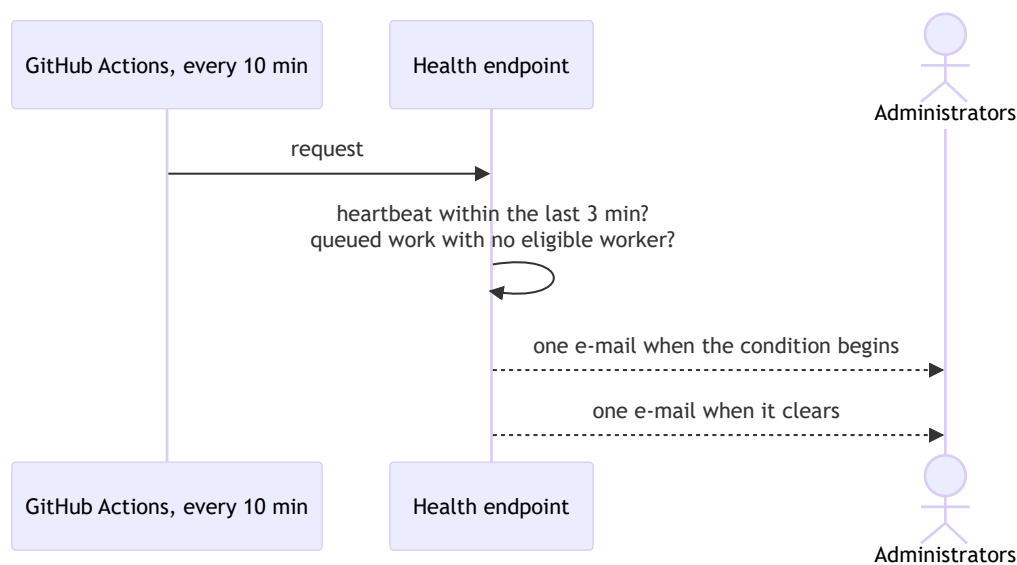


Figure 8.5 One alert per outage and one all-clear. The state is recorded in the activity log, so it is also visible on the site.

CHAPTER 9

Infrastructure

In this chapter. Where the software is deployed, how the website and the worker update themselves, how MATLAB is licensed inside a container, and how the system is run on a development machine.

Files to open: `provision-arbutus-worker.sh` → `vm-update.sh` → `Dockerfile.matlab` → `.env.example`.

9.1 The deployed services

The two deployed programs update themselves in different ways:

- **The website** deploys automatically on every push to the repository.
- **The worker** is a Linux virtual machine in the Alliance research cloud. It pulls the repository every ten minutes, restarts only when idle, and runs MATLAB inside a container licensed through the laboratory's MathWorks account.

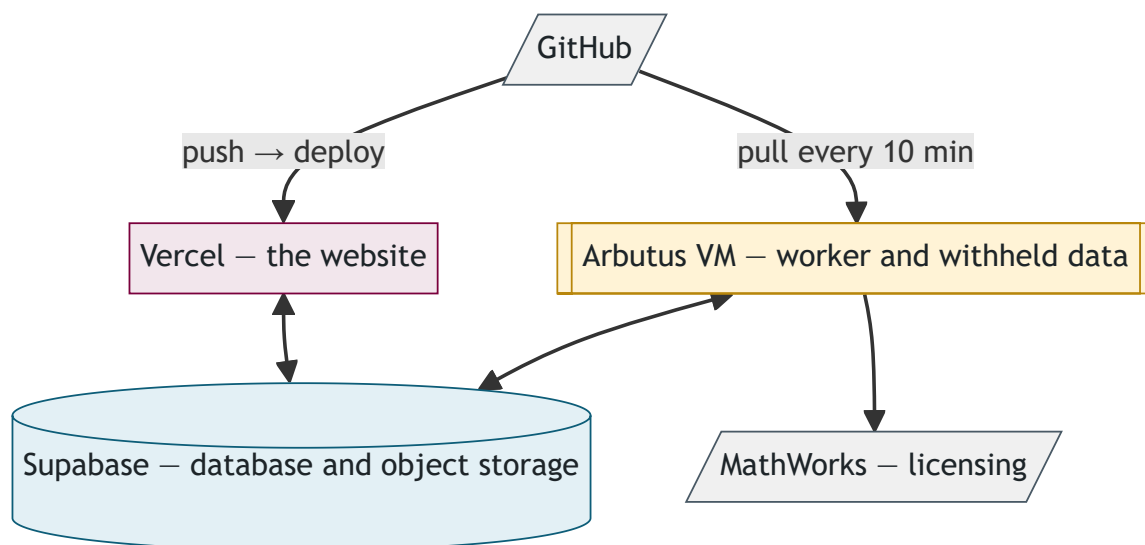


Figure 9.1 The three programs of Chapter 2 in their hosting context.

9.2 The virtual machine

A single provisioning script prepares the machine. It installs and configures:

- Docker, which provides the sandboxes;
- Node.js, the runtime for the worker;
- a service account with no interactive login, whose sole purpose is to run the worker;
- a checkout of the repository using a deploy key with read-only access;
- a hardened systemd service that keeps the worker running;
- a firewall that admits SSH and nothing else.

Secrets and the withheld data are placed manually after provisioning. They are owned by the service account and readable by no other user.

Self-update runs every ten minutes:

1. Fetch the repository.
2. If anything changed, rebuild only the affected components: dependencies, the database client, or the sandbox images.
3. Restart the worker, but only if no evaluation is in progress; otherwise defer to the next interval.

9.3 MATLAB in a container

MATLAB runs inside MathWorks' own container image and is licensed through the laboratory's MathWorks account rather than a licence server. A one-time interactive sign-in produced an identity token valid for one year, which is stored on the virtual machine. Before each evaluation the worker exchanges it for a 24-hour access token, and only that short-lived token is passed into the container, as Figure 9.2 shows:

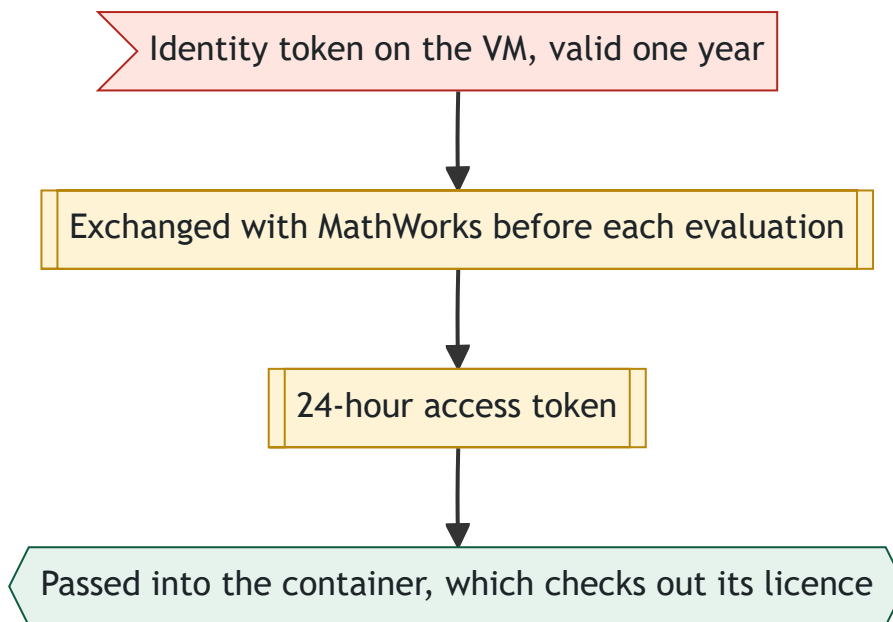


Figure 9.2 The licensing chain. The long-lived credential never enters the sandbox.

9.4 Continuous integration

A push that modifies the web tier triggers a browser test pass over the principal pages using Playwright, and the push is refused if any page fails to render. On success the site deploys automatically, and the virtual machine adopts the change within ten minutes.

9.5 Running the system locally

The same code runs on a development machine with different configuration. There is no simulated evaluator: a developer's worker runs the real evaluation pipeline, which requires the withheld data and the sandbox image. The repository README lists the five commands required, and the configuration differs from production in four respects:

- a local PostgreSQL instance in Docker,
- uploaded files stored on the local disk,
- e-mail delivered to a test inbox rather than to real addresses,
- a **seed** script that populates the empty database with one administrator account and one test user, and nothing else.

CHAPTER 10

Maintenance reference

The most common maintenance tasks and where each is performed. Anything not listed can be located from the *Files to open* entries of the preceding chapters.

TASK	PROCEDURE
Change a timeout or the daily submission limit	Admin → Evaluation workers. No deployment; effective within 15 s
Revise the scoring weights	Admin → Scoring weights: edit, preview, save with a justification, optionally notify authors
Change the scoring <i>computation</i>	Edit <code>pipeline.py</code> ; re-run the reference models and confirm that only the intended scores changed; increment the benchmark version in <code>socbench_eval/__init__.py</code> and <code>benchmark-version.ts</code> ; existing results become "legacy" and their authors are invited to resubmit
Add a metric	Add a column in <code>schema.prisma</code> , an entry in <code>test-cases.ts</code> , and its computation in <code>score()</code> ; the scorecard, PDF and CSV pick it up; confirm the weights still sum to 1
Add a worker machine	Provide the withheld data and <code>worker.env</code> , then <code>npm run worker</code> ; the machine registers itself and begins claiming jobs
Rotate a secret	Rotate at the source, update Vercel and <code>/etc/socbench/worker.env</code> , restart the worker
Renew the MATLAB licence (annually)	The Workers page shows the expiry; repeat the interactive sign-in and <code>matlab-mhlm-setup.sh</code>
Grant administrator rights	Admin → Users → Make admin, or add the address to <code>ADMIN_EMAILS</code> ; effective on the user's next request

CHAPTER 11

A ten-minute demonstration

1. **Leaderboard.** Rank, weighted error and complexity. Note that every figure is an error and that lower is better.
2. **One result.** The generated interpretation, the scorecard summing to the score, the worst-case trace, and the PDF report.
3. **Submission.** Run a test on the reference package (seconds), then submit it; observe the queue position and the progress indicator.
4. **Admin → Evaluation workers.** The machine that claimed the job: languages, load, console and licence expiry.
5. **Source, in this order.** `schema.prisma` → `claimJob` in `run-job.ts` → the `docker run` invocation in `python-evaluator.ts` → `score()` in `pipeline.py` → `Run_Model.m`.
6. **Conclude with the rule.** The web tier never executes submitted code and never has access to the withheld data.

BACK MATTER 12

Glossary

Battery terms

TERM	MEANING
SOC, state of charge	The fraction of usable capacity remaining, 0–100 %. It cannot be measured directly and is estimated from current, voltage and temperature.
BMS	Battery management system: the vehicle electronics that execute the estimator, one sample at a time.
Cell	One physical battery. Four Tesla 2170 cells are used, named for the simulated vehicle payload with which they were cycled: <code>m80</code> , <code>m448</code> , <code>m448N</code> , <code>m1000</code> . <code>m448</code> is withheld in its entirety.
Drive cycle	A standard speed profile converted to the current a cell delivers. UDDS (urban), HWFET (highway), LA92 and US06 (aggressive), HWCUST and HWGRADE (laboratory-designed, unpublished).
Open / withheld data	Open data is published for model development; withheld (blinded) data is reserved for scoring. <code>blind_data.mat</code> is the reference.
Coulomb counting	Integration of current over time: the simplest estimator, and the reference for the complexity scale.
EKF / UKF	Extended and unscented Kalman filters: model-based estimators that fuse a cell model with measurements.
FNN / LSTM / GRU / Transformer	Neural-network estimator architectures.
RMSE / MAE / maximum error	Root-mean-square, mean-absolute and worst single-sample error, in percentage points of SOC.
Test case	One of the eighteen scoring categories, each with a published weight.
Weighted error	The leaderboard score: the sum over test cases of weight × RMSE.

TERM	MEANING
Robustness run	A run with a deliberately incorrect initial SOC, or with a constant offset added to the current.
Padding	One hour of the first sample prepended to each cycle so that stateful models reach steady state.
Complexity	Execution time per sample relative to a Coulomb counter, binned 1–10. Informational only.
Test run (dry run)	An evaluation on open data from the submission page; not recorded on the leaderboard.
Package	The uploaded archive: <code>Model.py</code> or <code>Model.m</code> / <code>Model.p</code> together with parameter files.

Software terms

TERM	MEANING
Repository / git / GitHub	The source code with its full change history, hosted on GitHub.
Serverless	A hosting model in which a short-lived server instance handles each request and is suspended once it responds.
Container / Docker / image	An isolated execution environment; the runtime that creates it; the template from which it is created.
Sandbox	A container with the network disabled, a read-only filesystem and no privileges.
Database / table / row	PostgreSQL stores all persistent state as tables of rows.
Prisma	The typed database client; one schema file defines every table.
Server Component / Server Action	A page rendered on the server from the database; a form handler executed on the server. There is no separate API layer.
Signed URL	A pre-authorised, time-limited address that permits a browser to upload directly to object storage.

TERM	MEANING
Session token / JWT / cookie	A signed token held by the browser after sign-in, proving identity on each request.
bcrypt	A deliberately slow one-way hash function for passwords.
Rate limit	A cap on the number of attempts within a time window.
Queue / job / worker / heartbeat	Pending work; one unit of it; the process that performs it; the process's periodic liveness record.
Compare-and-swap	An update applied only if the row is unchanged since it was read; the mechanism by which two workers never claim the same job.
Environment variable / <code>.env</code>	Configuration and secrets supplied to a program from outside its code.
VM / Arbutus / systemd	A cloud virtual machine; the Alliance's cloud at the University of Victoria; the Linux service manager.
mode 600	A file readable and writable only by its owner.

Key figures

144 test cycles (4 cells × 6 temperatures × 6 cycles) · 195 runs per evaluation · 18 test cases with weights summing to 1 · 3 submissions per day and 5 test runs per hour · 6-hour evaluation limit · worker polls every 2 s, heartbeat every 15 s · a lock expires after 30 min, 2 attempts per job · 50 MB upload limit.

BACK MATTER 13

Appendix: reference tables

Tools

TOOL	ROLE	RATIONALE
TypeScript, Node.js	Language and runtime for the website and the worker	One typed language for both
Next.js 15, React 19	Web framework	Pages, forms and small APIs in one project; managed hosting
Tailwind CSS, Radix UI	Styling; accessible dialogs, menus and tooltips	Rapid development; McMaster colours as design tokens
Recharts, TanStack Table	Charts; the leaderboard's headless table	SVG charts with export; sorting and column selection
zod	Form validation	One rule set applied in the browser and on the server
Auth.js, bcryptjs	Sessions; password hashing	No third-party identity provider required
Prisma, PostgreSQL (Supabase)	Database client; the database	One schema file; managed tier with object storage
nodemailer, pdfkit	E-mail; the PDF report	Provider-independent mail; vector PDFs without a browser
Python, numpy, scipy	The evaluation pipeline	Exact, fast, licence-free scoring; reads <code>.mat</code> files
MATLAB R2026a	Executes <code>.m</code> / <code>.p</code> models only	Inside MathWorks' container image, licensed online
Docker	The sandbox	Isolation of untrusted code
Playwright	Browser tests	Executed on every push that modifies the web tier

TOOL	ROLE	RATIONALE
Vercel, Arbutus, GitHub Actions	Web hosting; the worker VM; the ten-minute health check	All on free or research tiers

Environment variables

GROUP	VARIABLES
Database	<code>DATABASE_URL</code> (connection pooler, port 6543), <code>DIRECT_URL</code> (port 5432, migrations only)
Website	<code>AUTH_SECRET</code> , <code>AUTH_URL</code> , <code>NEXT_PUBLIC_SITE_URL</code> , <code>ADMIN_EMAILS</code> , <code>CRON_SECRET</code> , <code>OPS_HEALTH_TOKEN</code>
Mail	<code>SMTP_HOST</code> , <code>SMTP_PORT</code> , <code>SMTP_USER</code> , <code>SMTP_PASS</code> , <code>MAIL_FROM</code> , <code>ADMIN_NOTIFY_EMAIL</code>
Storage	<code>STORAGE</code> (local / supabase), <code>UPLOAD_DIR</code> , <code>MAX_UPLOAD_MB</code> , <code>SUPABASE_URL</code> , <code>SUPABASE_SERVICE_KEY</code> , <code>SUPABASE_BUCKET</code>
Worker	<code>SOCBENCH_BLIND_DATA</code> , <code>SOCBENCH_PYTHON</code> , <code>MATLAB_BIN</code> , <code>WORKER_CONCURRENCY</code> , <code>WORKER_RUNTIMES</code>
Sandbox	<code>EVAL_SANDBOX</code> , <code>EVAL_SANDBOX_IMAGE</code> , <code>EVAL_SANDBOX_MATLAB_IMAGE</code> , <code>EVAL_MATLAB_MHLM_FILE</code> , <code>EVAL_MATLAB_NETWORK</code> , <code>EVAL_CPUS</code> , <code>EVAL_MEMORY</code>
Calibration	<code>SOCBENCH_CAL_PYTHON</code> , <code>SOCBENCH_CAL_MATLAB</code>

Every variable is annotated in `.env.example`. Production values exist only in Vercel's configuration and in `/etc/socbench/worker.env` on the virtual machine.

Limits

OPERATION	LIMIT
Registration	5 per hour per address
Sign-in	10 per 15 min per account, 40 per 15 min per IP address
Password reset / resend verification	3 per hour per e-mail address

OPERATION	LIMIT
Contact form	5 per hour per IP address
Upload URLs	30 per hour per user
Test runs	5 per hour (administrators unlimited)
Submissions	3 per rolling 24 h (administrators exempt)
Evaluation / test run	360 min / 10 min
Archive	≤ 500 entries, ≤ 512 MB unpacked, ≤ 256 MB per entry, ≤ 200 : 1 compression ratio, no sub-directories, no symbolic links

Administrative actions and their safeguards

ACTION	SAFEGUARD
Moderate a submission	Justification of at least 10 characters; not while RUNNING; contest entries cannot be made private (hide instead)
Bulk deletion	At most 100; RUNNING entries skipped; one activity-log entry; authors optionally notified
Delete a user	Not oneself; not an administrator (demote first); not while the user's work is RUNNING
Change a role	Not one's own
Pause, resume or stop a worker	Applied at the worker's next heartbeat
Release a job lock	Behind a confirmation; a live worker would evaluate the job twice
Evaluation settings	Timeout 10–1440 min, test run 2–60 min, submissions per day 1–100
Scoring weights	Preview required; must sum to 1
Contests	Only one open at a time

Principal columns

TABLE	COLUMNS OF NOTE
User	email , passwordHash , role , emailVerified (a timestamp), adminNotify preferences, avatar
Submission	seq (the visible number), modelName , modelType , runtime (python / matlab), status , version , isPrivate , isHidden , fileKey , contestId
EvaluationJob	attempts , lockedAt , lockedBy , log , cancelRequestedAt
EvaluationResult	weightedError , complexity , the 18 metric columns, maxError , perCycle , timeSeries , robustness , tracesKey , evaluatorVersion
ScoreRevision	kind (evaluation, failure, rescore, resubmission, edit, cancelled, legacy), the score and metrics at that moment, note , by
WorkerHeartbeat	hostname , lastSeenAt , runtimes , busyWith , paused , command , machine diagnostics, log
DryRun	its own status , lock and log ; result JSON; never touches withheld data, never on the leaderboard